CPU and GPU dataframe execution on one H100 host: a prospective blocked comparison of Polars and Spark

Daniel Ejiofor

Complete ten-block recovery analysis, 2026-10-28 UTC. The primary sample contains 400 fresh sessions under one fixed JVM profile. An exploratory pilot, an incomplete lost-host campaign and an aborted default-profile replacement have separate evidential roles; their timings are excluded from primary inference. A separate twenty-wave same-host contention extension is descriptive. Its observations neither replace nor augment the isolated primary sample.

Abstract

We compare Polars CPU, cuDF-Polars GPU, Spark CPU, and GPU-accelerated Spark on one H100 host, including compatible Polars 1.44.2 paths and a same-Python Polars 2.0 CPU control. Six selected queries derived from TPC-H run at five scales; six complex queries run at three scales. A default-profile replacement halted after 88 scientifically validated sessions when Spark CPU reported a Java heap-space error during a first warmup. Its timings are excluded from the fresh recovery sample. A separate feasibility diagnostic returned twelve oracle-valid outputs, and repeated SF1 readiness compared 116 successful outputs before one fresh 400-session recovery cohort began with a fixed Spark JVM allocation-retry profile. The design retains ten paired blocks, independent SQL correctness checks, separate action/common-host-result endpoints, and 26 primary comparisons with Holm adjustment. Strict unsupported expressions receive no imputed times. The profile is a post-incident configuration choice, with no causal mitigation or reliability claim. All 400 recovery sessions completed, with 9,360 oracle-valid outputs. All 26 prespecified tests reject after Holm adjustment: fifteen favor the GPU path and eleven favor its reference. At baseline SF300, CPU/GPU score ratios are 3.296 (pointwise 95% interval 3.239-3.355) for Spark, 3.537 (3.473-3.602) for Polars 1.44.2 CPU/Polars GPU and 2.451 (2.405-2.498) for Polars 2.0 CPU/Polars GPU. GPU Spark does not beat CPU Polars 2.0 on a complete-suite score at any sampled condition: their SF300 ratios are 0.698 (0.687-0.710) for baseline and 0.493 (0.479-0.507) for complex. Descriptive query results nevertheless favor GPU Spark for the window query and selected Q1/Q13 conditions. Strict Polars GPU supports four of six complex expressions and has no complete complex score. No available endpoint comparison changes point-estimate direction when measured through a common host result. A separate twenty-wave contention extension verifies 460 outputs and 110 common measured action pairs; it supplies no primary observations or causal slowdown estimate. These selected research workloads are not complete or certified TPC-H results.

1. Introduction

Dataframe users choosing between a CPU path and a GPU path face two connected questions: whether the selected expressions execute with the intended semantics, and whether a complete supported workload becomes faster at the data sizes they use. A fast subset cannot answer the complete-workload question when the remaining expressions are unsupported. Comparisons also depend on the timing boundary: process startup, materialization and conversion to a common host representation are distinct costs.

We study these questions for Polars CPU, strict cuDF-Polars GPU, Spark CPU and Spark with the RAPIDS plugin on one H100 host. The design compares five fixed software configurations, including two CPU Polars versions, on shared physical inputs. We ask where the prespecified contrasts support a GPU advantage, which unchanged expressions strict Polars GPU can translate, and which operators appear in final Spark plans. These are configuration comparisons; the same host does not isolate a causal effect of GPU hardware alone.

An exploratory H100 pilot motivated the comparisons and current CPU control. Its within-process repeats do not establish process-level replication. The prospective campaign retains the workload and collects fresh sessions in paired blocks, addressing repetition levels, uncertainty and setup bias. Kalibera and Jones, Mytkowicz et al.

The contribution is an auditable empirical comparison with four linked properties: fresh process-level replication in a balanced prospective schedule; independent SQL checks of every successful action; explicit expression coverage without imputed unsupported times; and separate primary action and descriptive common-host-result endpoints. The archive binds each claim to the exact source, inputs, schedule and raw artifacts that produced it. No novelty claim about GPU database architecture is needed for this bounded comparison to be useful.

The scope excludes distributed scaling, equal-cost performance, energy, controlled cold-cache latency and optimal per-engine tuning. Historical A100/H100 comparisons also change CPU allocation, software and file layout. Primary recovery conclusions apply to the explicitly recorded JVM profile; the aborted cohorts do not estimate its causal performance or reliability effect. The user-requested contention extension asks a different question about two coordinated workloads sharing the same host. It has separate sampling, instrumentation and descriptive reporting, and supplies no extra primary blocks.

2. Related work and methodological position

Systems benchmark repetition occurs at several levels. Kalibera and Jones distinguish variation between builds, executions and inner iterations, and argue that replication should address the level contributing uncertainty. Our inner actions summarize one fresh session; ten paired blocks supply the observations used for inference. We do not treat thirty inner actions as thirty independent sessions or claim that ten blocks guarantee a desired precision. Kalibera and Jones

Experimental layout can change measured conclusions even when the setup appears innocuous. Mytkowicz and colleagues demonstrate this problem and motivate setup randomization. Blocking provides a complementary way to organize comparisons around nuisance variation. Here, shared block/condition identities, randomized condition order and balanced configuration orders address allocation; they do not demonstrate constant host conditions or independence. Mytkowicz et al., NIST

The study fixes one family of 26 primary comparisons before observing their results and applies Holm adjustment. Query-level and endpoint sensitivity analyses have a different, descriptive role. The t intervals on block log ratios remain model dependent; resampling the same ten observations would not create further independent evidence. Hesterberg's discussion of small-sample bootstrap undercoverage is a reason to state these limits, rather than to present bootstrap computation as a replacement for replication. Holm, Hesterberg

Implementation documentation supplies a different kind of evidence. Polars GPU support, Spark AQE and the pinned RAPIDS compatibility notes explain why strict guards, final plans and explicit semantic rules are required. They cannot establish that a particular measured action used the intended path or returned a valid result; the saved session artifacts provide those checks. Likewise, TPC-H supplies workload

provenance, while this selected-query, fixed-parameter Float64 experiment has a narrower identity than the complete standard benchmark. Polars, Spark 3.5.9, RAPIDS 26.08.1, TPC-H 3.0.1

The recovery profile has a limited technical rationale. The OpenJDK allocation- starvation issue discusses GCLocker retries and possible longer stalls; it does not diagnose this study's failure. JEP 423 changes G1 JNI critical-region handling in JDK 22, while this experiment retains Java 17.0.20.1. The Spark proposal that used retry count 100 was closed unmerged, so it is not official Spark 3.5.9 tuning guidance. These sources motivate a separately gated feasibility diagnostic, not a claimed cure or measured causal benefit. OpenJDK JDK-8192647, JEP 423, Spark PR 51920

3. Materials and implementations

3.1 Host and software

The replacement Linux KVM host exposes one H100 80 GB HBM3 GPU, 26 Xeon Platinum 8480+ vCPUs and 237,490,843,648 RAM bytes, approximately 221 GiB. Its driver is 580.105.08 and kernel is 6.8.0-1046-nvidia. Archived readiness inventory records OpenJDK 17.0.20.1, CUDA driver API 13.0 and CUDA runtime 12.9. The CUDA 12 package closure is pinned; the driver's advertised compatibility level does not change the runtime version used by these environments. The lost host used driver 570.148.08 and a different kernel; its incomplete campaign cannot establish the replacement host's performance.

Configuration ID	Execution path	Principal version
polars_cpu_144	Polars CPU streaming	Polars 1.44.2
polars_cpu_200	Polars CPU streaming	Polars 2.0.0
polars_gpu	cuDF-Polars streaming, CPU fallback rejected	Polars 1.44.2; cuDF-Polars 26.10.2
spark_cpu	Spark CPU, local[26]	Spark 3.5.9
spark_gpu	Spark with RAPIDS plugin, local[26]	Spark 3.5.9; plugin 26.08.1

The frozen replacement inventories verify the same CPython 3.11.13 base executable/build (Clang 20.1.4) and matching common non-GPU package pins in both environments. GPU dependencies differ because the selected cuDF-Polars release requires older Polars. The historical CPU control used Python 3.10.12 and remains separate.

Spark retains the a100-parity settings, including a 96 GiB driver heap, 120 shuffle partitions, adaptive query execution, and UTC session time zone. GPU Spark uses two concurrent GPU tasks and a 2 GiB pinned-memory pool. The plan archives these settings and the plugin JAR SHA256; the complete audit verifies their effective values in every final session. Polars uses the allocated 26-thread setting without an equivalent fixed host-memory limit. Consequently, common hardware does not imply equal memory budgets. Spark AQE can change joins and partitioning using runtime statistics; final plans are retained for interpretation. Spark 3.5.9 documentation

The recovery applies the following sealed driver startup profile to both Spark configurations at every suite/scale:

```
SPARK_SUBMIT_OPTS=-XX:+UnlockDiagnosticVMOptions -XX:GCLockerRetryAllocationCount=100
-XX:+PrintFlagsFinal
```

Java 17.0.20.1, G1, the 96 GiB maximum heap and all listed query settings remain fixed. PrintFlagsFinal records the effective driver options before query timing. Competing Java-option variables must be absent;

the profile must not be injected through JAVA_TOOL_OPTIONS, which also affects Spark's command builder. EXECUTION_PROFILE.json binds the exact startup environment, Java executable/runtime files, launcher sources, Python environments and diagnostic/input/oracle evidence before recovery timing. Its exact SHA256 and pre-start identity are recorded in §4.1. The four repeated Spark readiness driver logs verify the profile, as do all 160 formal Spark driver logs and 24 idle-boundary profile receipts in the complete audit. Allocation-retry and GC stalls within an action remain inside its normal endpoints; no such observation is excluded as an outlier.

Polars GPU sets raise_on_fail=True, verifies the effective configuration, runs a positive GPU canary, and requires an unsupported CPU-only canary to raise. The streaming policy fallback_mode=warn concerns a possible fallback to a single GPU partition in the installed release; it does not authorize CPU translation fallback. Spark GPU may retain CPU operators. Installed source and guards define behavior; rolling documentation is contextual. Polars GPU support, pinned Spark plugin compatibility

3.2 Data and workload definitions

DuckDB 1.5.6's tpch extension generated the inputs, exported as Snappy Parquet under the archived recipe. Configurations share files and fixed manifests; full SHA256 checks run before/after the campaign outside measurements. Nominal scale factors are generator settings. Physical compressed bytes below describe all tables, not scan volume or query working set.

The default-profile replacement inputs were freshly generated using the pinned recipe. Their manifest and physical file identities remain the inputs for the recovery, which reuses the same 1,654 files and existing independent oracle bytes on the same VM. No recovery input regeneration or mutation is allowed. No claim of physical-byte identity with the lost instance is made. The archived replacement inventory is therefore also the recovery inventory below; the recorded pre/post full-byte gates verify those bytes. Generation uses 30 export threads, a 64 GB memory setting, Snappy compression, 122,880-row groups, per-thread output and a 128 MB target file size. Generation and export are outside competitor timings; timed configurations use 26 vCPUs.

SF	Line-item rows	Parquet files	Stored bytes, all tables	Suites
1	6,001,215	57	325,716,220	Baseline
10	59,986,052	123	3,442,803,044	Baseline
30	179,998,372	185	10,568,904,874	Baseline, complex
100	600,037,902	371	35,650,816,651	Baseline, complex
300	1,799,989,091	918	107,973,376,582	Baseline, complex

The baseline contains Q01, Q03, Q06, Q10, Q12, and Q14 adaptations. The complex-v1 suite contains a six-table Q9-derived join, Q13-derived outer join with nested aggregation, correlated semi/anti predicates, two window orderings, exact distinct counts, and a fact-table self-join with an inequality. Query parameters, deterministic output ordering, and SQL/native definitions are frozen. Selected monetary and quantity columns are converted to Float64 or DOUBLE, rather than preserving decimal financial semantics.

Fixed parameters, custom queries, omitted standard queries/refresh tests, and experimental timing distinguish this workload from the standard. DuckDB also documents generated-data differences. No QphH, official TPC metric or certification claim is made. DuckDB 1.5 extension documentation, TPC-H specification 3.0.1

Disk bytes exceeding GPU memory do not establish out-of-core capacity: projection, filtering, compression and intermediates determine working sets. Sampled peaks are not exact allocator maxima.

4. Prospective experimental design

4.1 Campaign identity, allocation and replication

The frozen experimental source is code commit c84ef6a07341f350dae6636ab709c1b65fd894e0, comprising a 52-file source snapshot. Four cohorts have distinct evidential roles. The exploratory pilot supplies motivation and historical observations. The October 26 prospective campaign was created at 2026-10-26 18:11:05 UTC with campaign SHA256 69669dfbc74ffb2a0c378c319c7b886043ffccb8f301784fb2b73b9ab32581e0; its host was lost before completion. Only three complete core checkpoints, 120 sessions and 2,808 successful output comparisons survive. It lacks final input-byte verification and a complete raw telemetry archive, and contributes no timings to either later primary sample.

The October 27 default-profile replacement used the same frozen source, design, seed, queries, settings and inferential method. Its campaign ID is h100-confirmatory-replacement-20261027, with campaign SHA256 0dabe0d40c5adda0a087508e3f95ceb0636815f5133a3b401071612e3f24c145 and pre-start manifest SHA256 a758ceea7d16a6c726221db5db2cb99502ac4f640131341b456e9b63ed459b2a. All 358 files in the pre-start archive were preserved off-host and verified; private Git commit b94372deebcf3503c115fd9cc396dec3666ce85b preceded the first confirmatory action. This archive includes fresh runtime inventories, all five input manifests, ten SF1 readiness sessions with 116 successful SQL output comparisons, five semantic fixture runs, and all eight independent full-scale oracle outputs. Readiness and oracle execution are outside the confirmatory performance sample. That cohort subsequently halted at 20:27:48 UTC with 88 scientifically validated sessions, 2,056 successful output comparisons and one failed session. Its remaining 311 sessions had not started. The complete abort and evidence disposition are reported in §5.1. No default-profile timing is carried into recovery inference, and its journal/attempts remain unchanged.

The fourth cohort, h100-confirmatory-gclocker100-20261027, began collection on October 27 at 21:47:44 UTC. Its canonical archive is evidence-research/20261027/h100-confirmatory-gclocker100-v1. The registered campaign SHA256 is 3142874b918a2df2d66c8dfb32c26f59a09353edcb4cb73e51c4ba46ff387641; the pre-start manifest SHA256 is ea57fa1221b905e8ee287fe7860285c7a37535b084f13ce3dbbbca5ac4e29ace; the sealed execution-profile SHA256 is a07569ffa3be9cd138ed63084ff59ed668b8bceeef355b5a4f63adef53464c69. The canonical pre-start snapshot contains 346 files, including its manifest; all 345 listed file checksums match. Private commit 6dbca2fbd4de1be8aa4408a639954bc9076bfa38 preceded the first session. The recorded full 1,654-file initial byte gate passed at 21:47:43.892782 UTC. The final full-byte gate passed at 2026-10-28 23:33:18.564788 UTC, after the last session ended. All 1,654 files retain their recorded identities.

Exactly one separately named full-complex-SF100 Spark CPU feasibility session completed before recovery preparation. It used seed 838471203, one warmup and one measured action per query. Its validation completed at 21:36:41.579958 UTC with all twelve oracle-valid records, twelve checked cleanup records, successful process exit and the five effective driver flags. The archive preserves diagnostic-sf100-driver_log.log, diagnostic-sf100-results.json and diagnostic-sf100-validation.json, with hashes bound by the sealed profile. Its timings are excluded from primary inference. This passed feasibility test does not establish mitigation efficacy or guaranteed robustness.

All five configurations then repeated SF1 readiness. The readiness report completed at 21:44:41.187289 UTC with ten smoke sessions, 116 successful SQL output comparisons, five semantic fixture configurations and no errors. All four Spark readiness driver logs show retry count 100, G1, unlocked diagnostics, PrintFlagsFinal and maximum heap 103,079,215,104 bytes. Passing before/after readiness profile receipts

bind the same sealed profile. These pre-start observations are excluded from the confirmatory performance sample.

The recovery retains the same VM, 1,654 physical files, existing oracle bytes, five configurations and unchanged 52-file C84 source. The fixed Spark JVM profile is an explicitly disclosed post-incident change. All ten blocks and 400 single-attempt sessions completed; the final replay binds the same profile and excludes every earlier timing cohort.

The design is ten blocks by eight suite/scale conditions by five configurations, yielding 400 fresh sessions. Each supported query has one recorded warmup, excluded from performance estimates, and three measured actions in a session. A fresh Python process or Spark JVM prevents reuse of the previous configuration's engine session, application table cache, or allocator. The three actions define a session median; they do not increase the independent inferential sample size beyond ten paired blocks.

For each condition, the ten sequences of an odd-five Williams design balance each configuration across positions and each ordered predecessor pair. The base sequence is [0,1,4,2,3]; cyclic relabelings and reversals supply the ten orders. Seed 20261026 fixes relabeling and sequence allocation. Condition order is randomized separately in each block. The five configurations within a block/condition share a recorded measured-query shuffle seed; warmup query order is fixed. Blocking and randomization reduce nuisance effects but do not prove independence or remove all temporal drift. NIST randomized-block guidance

The recovery requires all ten fresh blocks and all 400 fresh sessions. Ten blocks is a fixed allocation, not a power guarantee. No earlier timing enters that allocation, and there is no significance-based stopping, trimming, selective repetition or tuning. Unexpected errors halt execution; retained attempts and partial outputs support explicit deviation assessment. A recovery failure does not authorize automatic parameter escalation, restart or another cohort until a clean result appears.

4.2 Timing boundaries and host state

The primary `seconds` endpoint starts before query construction and ends after materialized `collect()`. It includes optimization, Parquet reading, execution, and result collection, while excluding process/session setup, table setup, plan serialization, sampler shutdown, and cleanup. Polars returns a host DataFrame; Spark returns Python Rows. The endpoint retains this API conversion asymmetry for continuity with the pilot.

Secondary `host_result_seconds` continues from the same start through Python dictionary conversion in both paths, excluding normalization/serialization. Endpoint direction changes are reported in §5.5. Startup, process wall time and action bounds are separate. A sum of six query medians is a suite score, not observed once-through latency.

All Parquet files at the selected scale are read before each session. Linux `mincore` snapshots are recorded immediately before/after this preheat and after the worker exits, together with memory, swap, faults, and disk counters. A residency snapshot cannot guarantee that pages remain resident during execution. The policy is attempted filesystem preheating with observed residency, not a controlled cold-cache or guaranteed in-memory benchmark. Application DataFrame caching is disabled.

Baseline has no explicit cleanup barrier; complex uses verified Python/JVM GC and active-job/shuffle checks outside action timers. Cleanup remains in process wall time. Sequential execution, exclusivity checks and process-group cleanup support control without establishing dedicated hardware or constant clocks. Plans show operator selection, not kernel/transfer times or runtime fractions. Additional profiling must remain outside primary timings.

Read-only status probes remain part of the instrumented virtual host. The replacement operational supervisor pauses after each complete 40-session block and snapshots all block raw files, including logs,

telemetry, events, plans, outputs and scientific gates. The next block requires an off-host receipt tied to the exact campaign and snapshot hashes. Transfers occur while no engine session is running. Backup pauses and receipt times are retained as an operational pacing amendment; they add no sessions and change no expressions, action boundaries or scheduled order. Full input-byte hashing, oracle execution and tests on the benchmark host remain outside engine measurements.

A separate local power event delayed the block06 off-host handoff on October 28. The controlling Mac’s selected power records show Low Power Sleep at 09:13:38 UTC on 1% battery and wake on AC attachment at 14:11:23 UTC. Local watcher observations bracket the gap at 09:13:25.168866 and 14:11:52.581715 UTC. Meanwhile, the remote block06 final session succeeded at 11:15:26.689677 UTC; its backup control was created at 11:15:30.821446 UTC and acknowledged off-host at 14:12:51.939753 UTC, 10,641.118307 s later. The existing controller and sleep-prevention process survived, and the remote supervisor continued the original schedule into block07 after the receipt, without restart or replacement. This local control/backup delay is distinct from the managed contention hold in §4.6 and supplies no GPU/JVM failure diagnosis. No action, worker or outer-wall timestamp is corrected. The longer between-block interval remains part of the host’s temporal/cache history; no performance effect is inferred from it.

The recovery retains 24 idle-boundary execution-profile receipts: before/after repeated readiness, before/after each of ten blocks, and before/ after final verification. The final complete audit binds those receipts to the sealed profile, unchanged Java/Python environments, session bounds and campaign completion. These new receipts are configuration checks, not extra timing observations.

4.3 Correctness and coverage

Before default-profile replacement timing, DuckDB 1.5.6 executed the original SQL definitions for all eight measured conditions using the same Float64 normalization. The recovery reuses those frozen complete oracle bytes. Each oracle contains complete ordered rows, SQL/source/manifest hashes, and effective resource settings. Its execution is outside competitor performance scores. This provides an independent execution engine for the specified SQL, not an independent proof that the SQL specification is correct.

Every successful warmup and measured result must match its frozen oracle: column sets, row counts, and ordered rows; exact nulls, strings, dates, and integer pairs; finite floating values within relative tolerance 1e-8 or absolute tolerance 1e-6. The audit retains numeric discrepancies. Parallel floating reductions may differ in operation order, so tolerance-based agreement does not imply bitwise identity or exact decimal arithmetic.

Strict Polars GPU rejection of the original Q13 DOTALL-regex expression and grouped-window expression remains explicit. Rejections have no imputed time. A complete suite score requires all six queries and all three measured actions to succeed and pass the oracle. Four supported complex queries cannot supply a six-query Polars GPU score. Coverage is part of the result.

Five separate hand-checked semantic cases cover duplicate/null join keys, outer counts, a nonempty semi/anti exclusion, rank ties/nulls, and multiline regex/distinct behavior. Their corrected diagnostic plans do not replace any performance query. A regression preserves a null-ordering mismatch observed in the original Polars 1.44.2 window port: with prices 100, 100, null, 50, its top-three keys are 3, 1, 2 instead of SQL’s 1, 2, 4. The earlier cohort’s full-scale input-domain checks find no null price/date/order/customer keys in the generated orders table at all five scales. The default-profile replacement repeated these SQL counts against all five fresh inputs before confirmation and likewise found zero nulls in those fields. Original-port claims are bounded to the archived tested domain; the adversarial limitation remains disclosed.

4.4 Estimands, uncertainty, and multiplicity

For block b, configuration e, and query q, let m[b,e,q] be the median of the three measured primary action times. A complete-suite score is S[b,e] = sum_q m[b,e,q]. For a prespecified reference/GPU comparison, L[b] = log(S[b,reference] / S[b,GPU]). The reported speedup estimate is exp(mean_b L[b]). Thus it is the geometric mean of paired block score ratios, rather than a ratio of pooled times or an arithmetic mean speedup. Ratios above one favor the denominator GPU configuration.

Two-sided pointwise 95% intervals are obtained by exponentiating mean(L) ± t(0.975,9) × sd(L)/sqrt(10). The corresponding two-sided tests evaluate mean log ratio zero. The model assumes independent, approximately normal block log ratios. Raw ratios, order and spread support assessment. Ten blocks may leave wide uncertainty; zero observed variance cannot prove reproducibility. Inner actions or resamples add no independent observations; small-sample bootstrapping is not an automatic remedy. Hesterberg

Primary comparison, numerator / denominator	Conditions	Tests
CPU Polars 2.0 / GPU Spark	All eight suite/scale conditions	8
CPU Spark / GPU Spark	All eight suite/scale conditions	8
CPU Polars 1.44.2 / Polars GPU	Five baseline scales	5
CPU Polars 2.0 / Polars GPU	Five baseline scales	5

The 26-test family uses Holm adjustment at familywise alpha 0.05. A supported GPU advantage requires an estimated ratio above one and adjusted $p < 0.05$, conditional on the model assumptions. Pointwise intervals are not simultaneous intervals. Lack of a supported difference does not establish equivalence. Failed/incomplete comparisons remain explicit and do not silently shrink the family. Query-level and host-result estimates are descriptive secondary results, with pointwise intervals and no added primary claims. Holm

Crossover statements identify only sampled scale points. The protocol’s supported-bracket rule requires a supported advantage at the upper measured point without a contradictory larger sampled point. A lower point lacking support is not evidence of reversal. No exact byte threshold or interpolation through unmeasured scales is asserted.

4.5 Separate coordinated same-host contention experiment

The owner subsequently requested concurrent sessions on the existing H100. The separate contention protocol defines h100-contention-20261028, with canonical pre-start archive evidence-research/20261028/h100-contention-v1. Its plan SHA256 is 670f37af6c1a6f2dbea5fc110d213b148a0acb83858786a21add8252acc89031 and pre-start manifest SHA256 is 4088b5302a7d42c8aa87ebaed06e229bed3b0783bd15cf6556a6110981bf9f25. All 65 listed file checksums match, for 66 canonical files including the manifest. Private pre-start commit 98c7ff4d6a9bbe82a021671c28fed28f71827b70 preceded execution. The plan binds the recovery campaign/profile identities above, the same 52-file C84 source, SF30/SF100 manifests, four existing independent oracles and separately versioned external instrumentation. No contention observation enters the isolated 400-session sample, its 26-test family or the recovery feasibility diagnostic.

Pair	Concurrent participants	GPU path
A	Spark GPU and Polars CPU 2.0.0	Spark GPU
B	Spark CPU and strict Polars GPU 1.44.2	Polars GPU

Each pair has one Spark JVM with its original 96 GiB heap and one GPU execution path. Each participant requests its full original 26 CPU threads on the same 26-vCPU host; CPUs are not partitioned into disjoint allocations or cgroups. Both Spark paths retain the sealed retry-100 JVM profile and fixed SQL settings. Host memory, CPU, cache, disk and GPU-related host work are deliberately shared. The design excludes two concurrent Spark heaps and simultaneous Spark GPU/ Polars GPU default pools. These observations cannot describe equal-resource competition or standalone isolated execution.

The staged pilot contains both pairs × baseline/complex at SF30: four waves and eight fresh processes, with one warmup and one measured action per query. Seed 20261028 fixes its wave order. Only if all four pilot waves pass output, cleanup, configuration and actual measured-overlap gates may the planned expansion run: both pairs × both suites × SF30/SF100 × two repetitions, sixteen waves and thirty-two fresh processes, with order seed 20261029. Pilot observations are not pooled into the expansion. Expected successful output comparisons are 92 for the pilot and 368 for the expansion, with two and eight expected strict unsupported warmups respectively. These are the prespecified coverage counts; actual audit accounting is reported in §5.7. Any failed wave stops its phase and preserves both participants and partial evidence. The design permits no hidden retry, adaptive sample increase or new scale; a failure requires a new explicit scientific decision.

The separately hashed `contention_worker.py` imports the byte-identical frozen worker. External hooks add an engine-ready barrier and a rendezvous before each common query/phase/trial. The action rendezvous follows the first existing process-counter sample and precedes the original timer start; both participants use the same future monotonic release timestamp. Query expressions, materialization/conversion code and timer statements remain unchanged. This is coordinated same-query contention, rather than independent arrival traffic. Each wave shares one full selected-scale preheat before either process starts, with residency observations before/after preheat and after both workers. Selected input-byte gates bracket each phase; these observations do not guarantee in-query residency.

In pair B complex, strict Polars GPU rejects `cx_q13` and `cx_window` during warmup. Both warmup attempts rendezvous, but their measured CPU counterparts explicitly bypass the barrier because no measured GPU counterpart exists. Those two CPU measurements are unpaired diagnostics. A six-query CPU sum would mix paired and unpaired conditions and must not be presented as uniformly contended execution; a complete six-query Polars GPU score remains unavailable. Only the four common measured query pairs establish query-level contention for this condition. Other waves require six common measured query pairs.

Saved action UTC and same-host monotonic intervals, rather than overlapping process lifetimes, must establish actual action overlap. Report both action durations, start skew and interval intersection for every common measured query; the pilot gate requires positive intersection on both clocks. The action and common-host-result endpoints retain their frozen boundaries. Wave/process wall time includes initialization and rendezvous waits, with shared preheat reported separately. Process-counter deltas and sampled RSS/GPU-memory observations can include untimed rendezvous waits and are not isolated action-only resource costs. Separate wave telemetry samples host/process trees every 0.5 s and the GPU board every 200 ms; the worker samplers retain their separately recorded intervals. Transient peaks may be missed, and board utilization/power cannot be attributed causally to a single query or process.

No contention p values, superiority decisions, additions to the primary family or exact crossover thresholds are declared. Two expansion repetitions support descriptive reporting only. Any later comparison with the completed isolated study must disclose the different replication, barrier and preheat context and keep both designs’ observations separate.

4.6 Managed interlude in isolated collection

The interlude record discloses a requested operational amendment after isolated collection began. The manager held only the exact isolated scheduler, PID 179144, at 2026-10-28 04:44:23 UTC, preserving its process, locks and journal. The already launched isolated SF30 Spark CPU runner and its workers continued unpaused. ORIGINAL_RUNNER_DRAINED.json records runner exit zero and schedule/result hashes at 04:45:43.946178 UTC, before the pilot start at 04:47:43 UTC. The frozen original action/host-result timer code is unchanged. The terminal offline audit now binds all 34 original block-04-complex-sf30-spark_cpu/attempt-001 files from the protected off-host block04 backup. It checks 33 immutable payload files byte for byte and binds session.json to its frozen identity while allowing only documented parent finalization; the later SCIENTIFIC_VALIDATION.json is retained separately. Its original result and schedule hashes match the drain receipt. The last original host-result endpoint is 04:45:38.505969 UTC; the first contention action starts at 04:47:59.693693 UTC, 141.188 s later. Saved action endpoints therefore establish temporal separation independently of the operational drain receipt.

Because the scheduler is held while its existing child drains, the journal may continue to mark the attempt as running after its worker has finished. The scheduler’s ended_at_utc and orchestrator_wall_seconds include the hold. The retained attempt reports an outer wall of 3,043.015 s and an end timestamp of 05:32:20.060856 UTC, while its Spark worker process wall is 240.581 s, ending at 04:45:41.907405 UTC. The outer envelope includes the scheduler hold and must not be interpreted as worker or query latency. These original values are preserved. The hold does not create a replacement attempt or cancel the remaining 400-session allocation.

The saved handoff records the same scheduler PID 179144, creation time 1793160829.75, resumed at 05:32:20.044724 UTC. STATE.json is COMPLETE_RESUMED; the phase driver is COMPLETE_OFFHOST, both phase raw backups are verified off-host, and the owned-group cleanup receipt lists no live members. PROFILE_BEFORE_RESUME.json records PASS at 05:32:19.993450 UTC. These interlude checks are separate receipts and do not replace or augment the formal 24-record profile log. The original unchanged schedule continues. The last contention host-result endpoint is 05:28:37.930202 UTC, before this resume. The terminal handoff audit reports PASS, verifies the original payload and action separation, and records no unresolved reasons. Temporal separation does not remove the interlude’s change to pacing or demonstrate unchanged host/cache conditions; possible carryover remains a limitation.

5. Results

All primary estimates below come from the complete fresh recovery cohort. The genuine portable replay is PASS, phase complete and nonsynthetic; no pilot, lost-host, aborted replacement, readiness, diagnostic or contention timing enters its ten-block estimates. The primary tables retain all prespecified contrasts, including supported reference advantages.

5.1 Cohort disposition and completed recovery

The default-profile replacement halted at 20:27:48 UTC on October 27. Its failed session, block-03-complex-sf100-spark_cpu, failed in cx_q09's first warmup, before any successful output or measured observation in that session. The JVM exited 52 and reported java.lang.OutOfMemoryError: Java heap space, preceded by GCLocker allocation-retry warnings. The event-log BlockManager maximum on-heap value, 61,658,785,382 bytes, corroborates the configured 96 GiB heap cap. The nearest host sample recorded 211,723,608,064 available RAM bytes, zero swap use and 2,713,801,965,568 free disk bytes.

These observations do not reveal failure-time heap occupancy or RSS and do not distinguish genuine heap exhaustion from premature allocation failure. No failure-time heap/RSS or GC-recovery evidence was retained, so a root cause and a causal effect of increasing the retry count remain undetermined. Host available memory alone cannot establish whether this JVM could satisfy its particular allocation.

The aborted cohort retains 88 scientifically validated sessions and 2,056 successful output comparisons, one failed session and 311 unstarted sessions. Its complete 3,349-file incident evidence was preserved off-host as a 220,656,964-byte gzip archive with SHA256 79c99819d272cc1af947d8e200976a940cd63a96be580112e4a6c5900adee6ff. Both complete blocks also retain separate verified off-host raw backups. The original journal and attempts were not edited; no successful session was rerun within that cohort, and its failed attempt remains visible. This is a reported failure observation, not a complete primary comparison.

The separate feasibility/readiness results in §4.1 preceded the fresh recovery sample. Its 400 sessions all completed in one successful attempt each, with no replacement observations. Recorded attempt bounds are 2026-10-27 21:47:44.042050 UTC through 2026-10-28 23:26:10.952925 UTC. The final full-input hash gate passed at 23:33:18.564788 UTC and journal completion was recorded at 23:33:18.565015 UTC. The original supervisor exited zero, with its end marker at 23:42:11 UTC. All ten 40-session block backups and final/completion snapshots were verified off-host. The separate managed contention hold and local-power handoff delay are retained as operational pacing disclosures, not removed from saved timestamps.

The independently replayed complete audit reports PASS with no findings. It verifies 400 session identities, 9,360 successful SQL comparisons, 60 specific strict translation rejections, 3,360 successful complex cleanup records and 60 failed-action cleanup records. All 160 formal Spark driver logs prove the sealed retry-100/G1/96 GiB profile, and all 24 ordered idle-boundary receipts match the registered Java/runtime/environment identities. This is successful execution under the chosen profile, not a causal before/after mitigation study.

Evidence role	Observed disposition	Role in primary inference
Exploratory H100 pilot	40 historical sessions; 936 saved successful outputs later checked	Historical context only
Lost October 26 host	120 core-checkpoint sessions/2,808 outputs survive; full campaign unavailable	Excluded
Default-profile replacement	88 validated sessions/2,056 outputs, one failed session, 311 unstarted	Excluded
Fixed-profile recovery	400 single-attempt sessions, ten blocks, complete audit and replay PASS	Entire primary sample
Separate feasibility/readiness	Twelve diagnostic outputs; 116 repeated readiness outputs	Excluded
Separate contention extension	Twenty waves, 460 successful outputs, terminal audit COMPLETE	Descriptive only

5.2 Correctness, coverage and complete-suite scores

All 9,360 successful recovery outputs match the independent SQL references under the declared rules: 2,340 warmups and 7,020 measured actions. The 60 expected outcomes are strict Polars GPU rejections of the original complex Q13 and grouped-window expressions, once per query in each of thirty complex sessions. They are retained translation limitations, not successful outputs or imputed measurements. No unexpected error or oracle failure is accepted by the completed audit.

Table 1 retains all forty configuration/condition cells. Every numeric cell has ten complete blocks and six of six supported queries. Each Polars GPU complex N/A cell has zero complete six-query scores out of ten planned blocks, with four of six original queries supported. The score is the geometric mean of ten session sums of query medians, in seconds; it is not observed latency for one sequential suite execution. All configurations retain their recorded software versions and resource policies.

Table 1. Complete-suite scores and coverage. CPU144 and CPU200 denote Polars CPU 1.44.2 and 2.0.0; PGPU is strict Polars GPU 1.44.2; SCPU and SGPU are Spark CPU and GPU. Numeric cells: 10/10 complete blocks, 6/6 queries. N/A cells: 0/10 complete blocks, 4/6 queries.

Suite/SF	CPU144 (s)	CPU200 (s)	PGPU (s)	SCPU (s)	SGPU (s)
baseline/1	0.164	0.152	0.408	2.002	2.718
baseline/10	1.107	0.950	0.742	6.833	4.470
baseline/30	3.659	2.800	1.433	17.273	7.206
baseline/100	12.144	9.039	3.810	47.633	15.273
baseline/300	40.579	28.122	11.473	132.767	40.275
complex/30	13.276	4.961	N/A	42.234	10.623
complex/100	51.149	19.657	N/A	132.131	37.523
complex/300	161.222	65.336	N/A	345.570	132.625

Complete-case scores and original-query coverage

Single H100 study host · fixed configuration · four engine paths, five software configurations · primary action endpoint

Condition	CPU Polars 1.44.2	CPU Polars 2.0.0	Polars GPU 1.44.2 / cuDF 26.10.2	CPU Spark 3.5.9	GPU Spark 3.5.9 / RAPIDS 26.08.1
Baseline SF1	0.164 s 10/10 blocks · 6/6 queries	0.152 s 10/10 blocks · 6/6 queries	0.408 s 10/10 blocks · 6/6 queries	2.002 s 10/10 blocks · 6/6 queries	2.718 s 10/10 blocks · 6/6 queries
Baseline SF10	1.107 s 10/10 blocks · 6/6 queries	0.950 s 10/10 blocks · 6/6 queries	0.742 s 10/10 blocks · 6/6 queries	6.833 s 10/10 blocks · 6/6 queries	4.470 s 10/10 blocks · 6/6 queries
Baseline SF30	3.659 s 10/10 blocks · 6/6 queries	2.800 s 10/10 blocks · 6/6 queries	1.433 s 10/10 blocks · 6/6 queries	17.273 s 10/10 blocks · 6/6 queries	7.206 s 10/10 blocks · 6/6 queries
Baseline SF100	12.144 s 10/10 blocks · 6/6 queries	9.039 s 10/10 blocks · 6/6 queries	3.810 s 10/10 blocks · 6/6 queries	47.633 s 10/10 blocks · 6/6 queries	15.273 s 10/10 blocks · 6/6 queries
Baseline SF300	40.579 s 10/10 blocks · 6/6 queries	28.122 s 10/10 blocks · 6/6 queries	11.473 s 10/10 blocks · 6/6 queries	132.767 s 10/10 blocks · 6/6 queries	40.275 s 10/10 blocks · 6/6 queries
Complex SF30	13.276 s 10/10 blocks · 6/6 queries	4.961 s 10/10 blocks · 6/6 queries	N/A 0/10 complete · 4/6 queries	42.234 s 10/10 blocks · 6/6 queries	10.623 s 10/10 blocks · 6/6 queries
Complex SF100	51.149 s 10/10 blocks · 6/6 queries	19.657 s 10/10 blocks · 6/6 queries	N/A 0/10 complete · 4/6 queries	132.131 s 10/10 blocks · 6/6 queries	37.523 s 10/10 blocks · 6/6 queries
Complex SF300	161.222 s 10/10 blocks · 6/6 queries	65.336 s 10/10 blocks · 6/6 queries	N/A 0/10 complete · 4/6 queries	345.570 s 10/10 blocks · 6/6 queries	132.625 s 10/10 blocks · 6/6 queries

Values are descriptive geometric means of ten complete session scores in seconds. Each score sums six query medians, not an observed whole-suite latency. Every cell reports planned-block completeness and query coverage. Polars GPU complex retains four of six supported original queries and has zero complete six-query scores; a partial four-query sum is never substituted. Versions are the fixed study configuration; the renderer does not independently inventory hardware or runtimes.

Campaign: h100-confirmatory-gclocker100-20261027 · analysis SHA256 3086aa09e7951693

Values are descriptive geometric means of ten complete session scores in seconds. Each score sums six query medians, not an observed whole-suite latency. Every cell reports planned-block completeness and query coverage. Polars GPU complex retains four of six supported original queries and has zero complete six-query scores; a partial four-query sum is never substituted. Versions are the fixed study configuration; the renderer does not independently inventory hardware or runtimes.

5.3 All primary contrasts

All twenty-six two-sided tests reject after Holm adjustment across the one prespecified family: fifteen favor the denominator GPU path and eleven favor the numerator reference. The largest adjusted p value is 5.545e-6. No contrast has zero observed log-ratio variance. Every estimate uses ten paired blocks; its displayed 95% interval is pointwise and is not a simultaneous confidence statement. The supported direction comes from the ratio and the Holm-adjusted test, not from a separate unadjusted-interval rule.

Tables 2a-2d give the complete family. B denotes baseline and C denotes complex; each row's exact contrast ID is its heading's code prefix followed by -baseline-sf<scale> or -complex-sf<scale>. Ratios are reference/GPU, so values above one favor GPU execution. CPU denotes the specific numerator reference, not a pooled CPU class. Values are rounded for display; full precision is retained in the bound JSON/CSV artifacts.

Table 2a. CPU Polars 2.0 / GPU Spark. Prefix polars_cpu_200_over_spark_gpu.

Condition	Ratio	Pointwise 95% CI	Raw p	Holm p	Supported path
B/1	0.055945	[0.054808, 0.057105]	1.53980e-19	4.00348e-18	CPU reference
B/10	0.212549	[0.208690, 0.216479]	1.48899e-17	3.57358e-16	CPU reference
B/30	0.388511	[0.382310, 0.394813]	3.92243e-16	7.84485e-15	CPU reference
B/100	0.591820	[0.580946, 0.602897]	2.80633e-13	2.52570e-12	CPU reference
B/300	0.698245	[0.686991, 0.709683]	2.56562e-12	1.79594e-11	CPU reference
C/30	0.467006	[0.431938, 0.504921]	3.80945e-09	1.14283e-08	CPU reference
C/100	0.523867	[0.515150, 0.532732]	1.74612e-14	2.26996e-13	CPU reference
C/300	0.492635	[0.478992, 0.506668]	7.89815e-13	6.31852e-12	CPU reference

Table 2b. CPU Spark / GPU Spark. Prefix spark_cpu_over_spark_gpu.

Condition	Ratio	Pointwise 95% CI	Raw p	Holm p	Supported path
B/1	0.736771	[0.685003, 0.792450]	5.54466e-06	5.54466e-06	CPU reference
B/10	1.528723	[1.451437, 1.610125]	1.79765e-08	3.59529e-08	GPU
B/30	2.396972	[2.300420, 2.497576]	3.63646e-12	2.18188e-11	GPU
B/100	3.118779	[3.055933, 3.182918]	6.16814e-16	1.11026e-14	GPU
B/300	3.296495	[3.239099, 3.354908]	1.06637e-16	2.34602e-15	GPU
C/30	3.975933	[3.678199, 4.297766]	1.85028e-11	7.40110e-11	GPU
C/100	3.521337	[3.472958, 3.570391]	7.66732e-18	1.91683e-16	GPU
C/300	2.605614	[2.556563, 2.655606]	1.56222e-15	2.65578e-14	GPU

Table 2c. CPU Polars 1.44.2 / Polars GPU. Prefix polars_cpu_144_over_polars_gpu.

Condition	Ratio	Pointwise 95% CI	Raw p	Holm p	Supported path
B/1	0.401941	[0.390084, 0.414158]	1.45161e-13	1.45161e-12	CPU reference
B/10	1.491644	[1.474820, 1.508659]	3.87911e-14	4.26702e-13	GPU
B/30	2.554085	[2.513373, 2.595456]	4.17276e-16	7.92824e-15	GPU
B/100	3.187462	[3.125726, 3.250418]	3.62900e-16	7.62090e-15	GPU
B/300	3.536788	[3.473146, 3.601596]	8.58767e-17	1.97516e-15	GPU

Table 2d. CPU Polars 2.0 / Polars GPU. Prefix polars_cpu_200_over_polars_gpu.

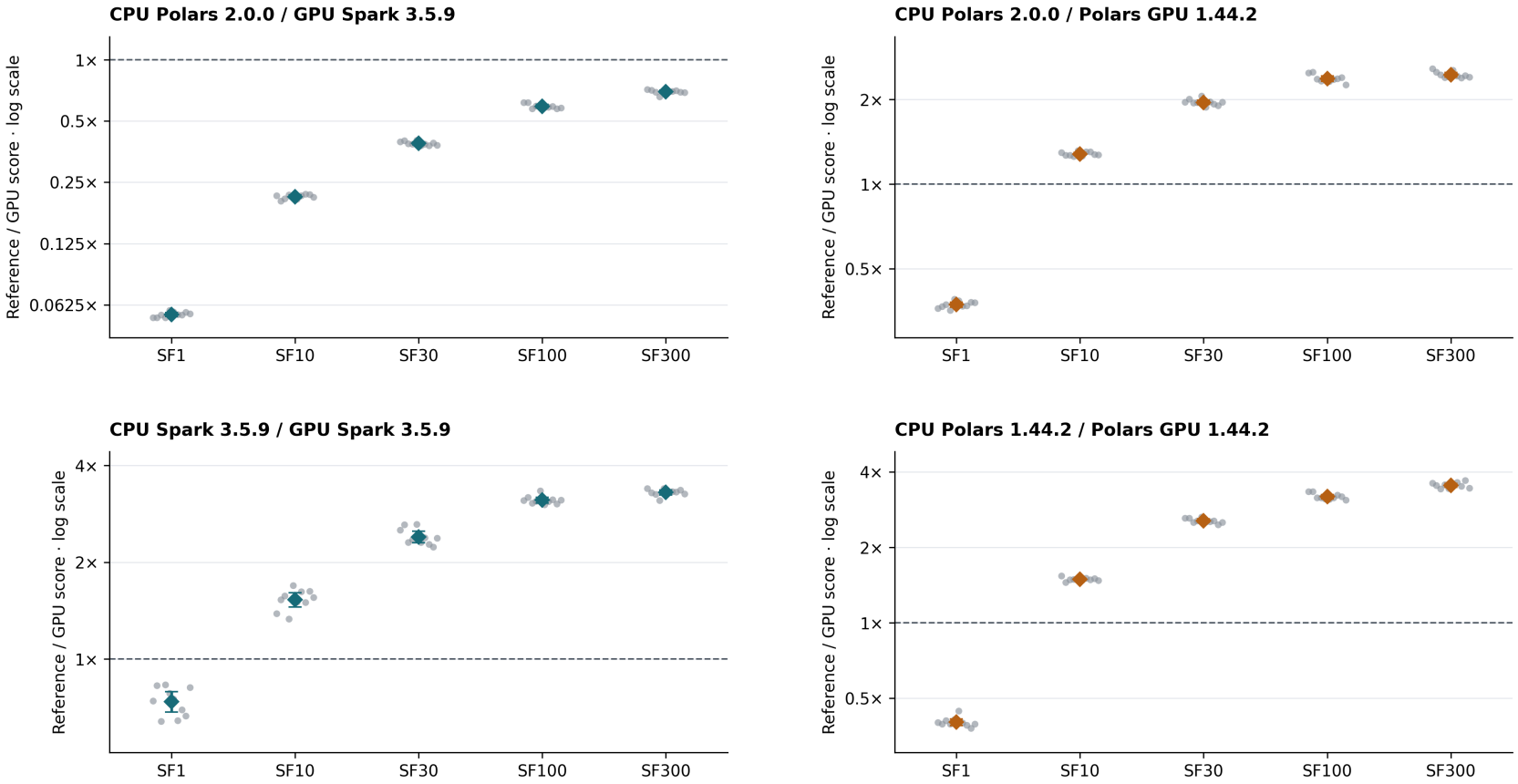
Condition	Ratio	Pointwise 95% CI	Raw p	Holm p	Supported path
B/1	0.372908	[0.365387, 0.380583]	2.23869e-15	3.58190e-14	CPU reference
B/10	1.280664	[1.265771, 1.295732]	3.81639e-12	2.18188e-11	GPU
B/30	1.954230	[1.918831, 1.990282]	2.73543e-14	3.28252e-13	GPU
B/100	2.372428	[2.320547, 2.425469]	1.53940e-14	2.15516e-13	GPU
B/300	2.451064	[2.405104, 2.497903]	2.72844e-15	4.09266e-14	GPU

GPU Spark has a supported complete-suite advantage over CPU Spark at baseline SF10, SF30, SF100 and SF300, and at all three complex scales. CPU Spark is supported at baseline SF1. Against CPU Polars 2.0, GPU Spark has no supported complete-suite advantage in any of the eight sampled conditions; CPU Polars 2.0 is supported in all eight. At SF300 their ratios are 0.698245 for baseline and 0.492635 for complex.

Strict Polars GPU has a supported baseline advantage over each CPU Polars version at SF10, SF30, SF100 and SF300. Both CPU versions are supported at SF1. At SF300 the ratio is 3.536788 against CPU 1.44.2 and 2.451064 against CPU 2.0. The choice of CPU control therefore changes the reported GPU gain. Neither CPU version supplies a complete complex-suite comparison with Polars GPU.

Baseline suite · measured scale points

Single H100 study host · fixed configuration · four engine paths, five software configurations



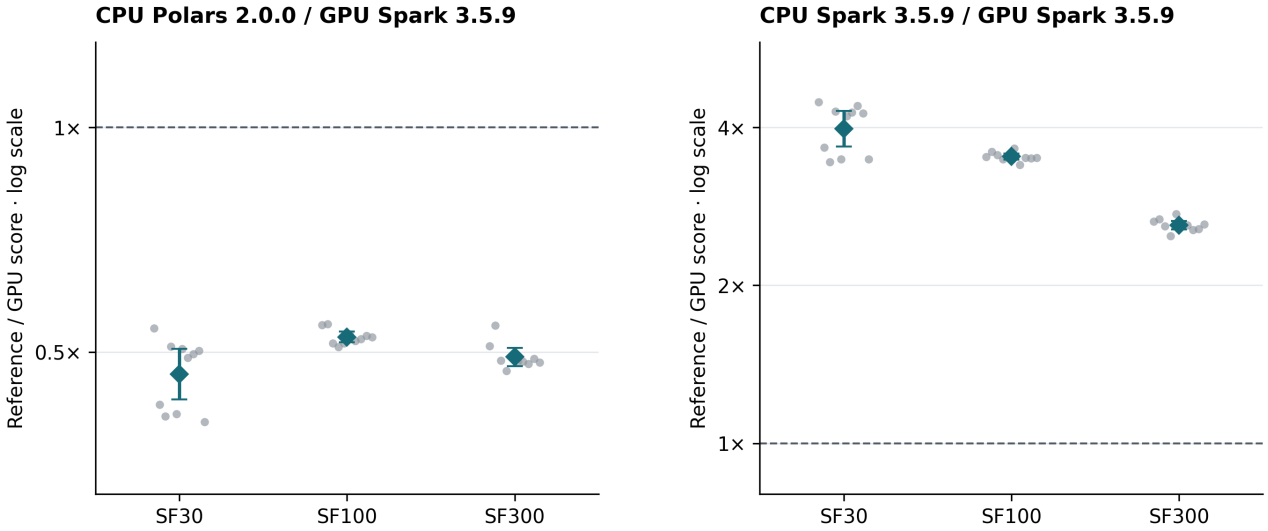
Each small point is one of ten paired block ratios. Diamonds and bars show the geometric mean and pointwise 95% Student-t interval on block log ratios; intervals are not simultaneous. The 26 prespecified suite tests form one Holm family; adjusted p values are in primary_contrasts.csv. Ratio >1 favors the GPU. Each session score sums six query medians from three inner actions; this is not an observed suite latency. Action time excludes startup. Blocks, rather than inner actions, are the inference units. Independent, approximately normal block log ratios are assumed; ten blocks do not guarantee power.

Campaign: h100-confirmatory-gclocker100-20261027 · analysis SHA256 3086aa09e7951693

Each small point is one of ten paired block ratios. Diamonds and bars show the geometric mean and pointwise 95% Student-t interval on block log ratios; intervals are not simultaneous. The 26 prespecified suite tests form one Holm family; adjusted p values are in primary_contrasts.csv. Ratio >1 favors the GPU. Each session score sums six query medians from three inner actions; this is not an observed suite latency. Action time excludes startup. Blocks, rather than inner actions, are the inference units. Independent, approximately normal block log ratios are assumed; ten blocks do not guarantee power.

Complex suite · complete comparisons and coverage

Single H100 study host · fixed configuration · four engine paths, five software configurations



Polars GPU complex complete suite: N/A

4 of 6 original queries supported at each measured scale.

Q13 regex: unsupported
Grouped window: unsupported

No six-query score;
no partial-sum comparison.

Each small point is one of ten paired block ratios. Diamonds and bars show the geometric mean and pointwise 95% Student-t interval on block log ratios; intervals are not simultaneous. The 26 prespecified suite tests form one Holm family; adjusted p values are in primary_contrasts.csv. Ratio >1 favors the GPU. Each session score sums six query medians from three inner actions; this is not an observed suite latency. Action time excludes startup. Blocks, rather than inner actions, are the inference units. Independent, approximately normal block log ratios are assumed; ten blocks do not guarantee power.

Campaign: h100-confirmatory-gclocker100-20261027 · analysis SHA256 3086aa09e7951693

Each small point is one of ten paired block ratios. Diamonds and bars show the geometric mean and pointwise 95% Student-t interval on block log ratios; intervals are not simultaneous. The 26 prespecified suite tests form one Holm family; adjusted p values are in primary_contrasts.csv. Ratio >1 favors the GPU. Each session score sums six query medians from three inner actions; this is not an observed suite latency. Action time excludes startup. Blocks, rather than inner actions, are the inference units. Independent, approximately normal block log ratios are assumed; ten blocks do not guarantee power.

5.4 Sampled-scale brackets and descriptive query heterogeneity

For baseline Spark CPU/GPU and both baseline CPU Polars/Polars GPU comparisons, SF1 favors the CPU reference and SF10 favors the GPU, with no contradictory larger sampled point. The supported sampled-scale bracket is therefore SF1 to SF10 for each of these three declared comparisons. It does not identify an exact intermediate threshold. GPU Spark never crosses CPU Polars 2.0 on the sampled complete-suite conditions. The complex Spark CPU/GPU comparison already favors the GPU at its smallest measured scale, SF30; no lower crossover bracket is observed. The complex Spark CPU/GPU ratio falls from 3.975933 at SF30 to 2.605614 at SF300, so the gain is not monotone increasing with input scale.

Complete-suite results do not apply uniformly to individual queries. GPU Spark has a shorter descriptive query estimate than CPU Polars 2.0 for baseline Q1 at SF30/SF100/SF300, complex window at all three measured scales, and complex Q13 at SF100/SF300. For example, at SF300 the CPU Polars 2.0/GPU Spark ratio is 5.145 for window but 0.149 for Q9. Table 3 shows all SF300 expressions; the heatmap retains all measured scales. These are geometric means of paired query ratios, without query-level significance tests or additional primary claims. The four supported complex Polars GPU expressions cannot establish a complete six-query score.

Table 3. SF300 descriptive action ratios, CPU Polars 2.0 / GPU path. Ratios above one have the shorter GPU point estimate. N/A denotes an original strict translation rejection and receives no imputed time.

Suite	Query	CPU200 / SGPU	CPU200 / PGPU
baseline	q01	2.259	3.924
baseline	q03	0.576	2.600
baseline	q06	0.603	1.162
baseline	q10	0.494	2.564
baseline	q12	0.417	1.940
baseline	q14	0.606	1.508
complex	cx_q09	0.149	3.603
complex	cx_q13	1.826	N/A
complex	cx_semi_anti	0.755	2.558
complex	cx_window	5.145	N/A
complex	cx_distinct	0.634	4.600
complex	cx_self_join	0.847	1.522

Query ratios · descriptive action endpoint

CPU Polars 2.0.0 / GPU · Single H100 study host · fixed configuration · four engine paths, five software configurations



Cells show geometric means of ten paired block query ratios for CPU Polars 2.0.0 / GPU, using the action endpoint. Color encodes log2(ratio), centered symmetrically on 1×. These query comparisons are descriptive; no query significance tests or multiplicity-adjusted query claims are made. Gray N/A cells retain the original strict GPU regex and grouped-window rejections. They are not a ratio of 1 and receive no imputed latency. The original correlated query does not establish an executed nonempty anti join.

Campaign: h100-confirmatory-gclocker100-20261027 · analysis SHA256 3086aa09e7951693

Cells show geometric means of ten paired block query ratios for CPU Polars 2.0.0 / GPU, using the action endpoint. Color encodes log2(ratio), centered symmetrically on 1×. These query comparisons are descriptive; no query significance tests or multiplicity-adjusted query claims are made. Gray N/A cells retain the original strict GPU regex and grouped-window rejections. They are not a ratio of 1 and receive no imputed latency. The original correlated query does not establish an executed nonempty anti join.

5.5 Common-host-result endpoint sensitivity

All thirty-two prespecified descriptive configuration/condition pairs are retained in Appendix B. Twenty-six have ten paired block observations at both endpoints; six Polars GPU complex comparisons remain unavailable. No available point estimate changes direction relative to the action endpoint. This agreement is specific to the saved outputs and conversion boundaries and does not establish that conversion costs are absent.

For example, CPU Polars 2.0/Polars GPU at baseline SF1 changes from 0.372908 (action) to 0.373832 (common host result); at SF300 it changes from 2.451064 to 2.451062. CPU Polars 2.0/GPU Spark complex SF300 changes from 0.492635 to 0.492637. Appendix B preserves each endpoint's pointwise interval; these secondary rows introduce no hypothesis tests or familywise superiority claims.

5.6 Block spread, temporal order and host state

Appendix C displays all 260 primary block ratios in chronological block order, without trimming or selective repetition. For every contrast the ten observed ratios are on the same side of one. This is a description of these observations, not proof of independence or a separate inferential test. Standard deviations of block log ratios range from 0.015856 to 0.109119. The largest is CPU Polars 2.0/GPU Spark complex SF30, whose raw ratios range from 0.403222 to 0.537869; Spark CPU/GPU Spark complex SF30 ranges from 3.439022 to 4.470605. The saved raw ratios make this variation visible beside the pointwise intervals.

The complete audit checks the required session configurations, cache inventories, cleanup records and input-byte chronology. Per-session preheat and instantaneous residency observations do not guarantee constant in-query residency or host clocks. No formal drift or carryover test is added. The managed contention hold in §4.6 and the distinct local low-power/backup gap in §4.2 change temporal spacing and possible later cache/host state. In the latter event, block06's backup creation-to-off-host-ACK interval was 10,641.118307 s; remote collection completed the block during the local gap. Original action, worker and outer-wall timestamps are preserved; no interval is subtracted to improve a performance estimate. The post-incident fixed profile and all recorded pacing conditions bound interpretation of the complete sample.

5.7 Separate descriptive contention results

Both planned contention phases completed their workload and phase validations:

Separate phase	Passing/planned waves	Successful outputs, warmup/measured	Common measured pairs	Unpaired CPU measured diagnostics
SF30 pilot	4/4	46/46	22	2
SF30/SF100 expansion	16/16	184/184	88	8

The independent terminal offline audit at 07:15:58.632271 UTC verifies all 460 SQL output comparisons: 230 warmups and 230 measured outputs, ten expected strict unsupported warmups, zero oracle failures and zero unexpected query errors. It also verifies 110 common measured action pairs with positive within-pair intersection on both clocks, 250 barrier releases and ten unpaired CPU measured diagnostics. These aggregate counts account for preserved artifacts; they do not pool pilot/expansion timing estimates or add primary observations. Pair B complex still has four common measured pairs and two unpaired CPU diagnostics per affected wave, with no uniformly contended six-query CPU sum.

Pilot and expansion raw archives are verified off-host, and the phase driver reports COMPLETE_OFFHOST. The expansion archive contains 1,561 payload files in 487,321,600 tar bytes, SHA256 98be99cf74690d01053de9670669e629a166b02bd67fd51a994425006f792197. The final handoff independently binds the actual original 34-file attempt and same-process scheduler resume described in §4.6. The terminal receipt has status: COMPLETE, integrity_status: PASS, publication_ready: true, no findings and no unresolved handoff reasons. This completes the separate extension's provenance gate; its sample remains separate from the completed 400-session study.

The terminal CONTENTION_REPLAY.json SHA256 is 0cad03b7ab2edc7e0d62dbcc7b84f8fa0ea51325aba41eb354b5baae0c51a2d3. The earlier 06:17 v3 receipt remains an unchanged historical PARTIAL audit: its then-missing original payload is now supplied to this new terminal audit. Its former no_action_overlap: false denoted then-unavailable proof, not an observed violation. The terminal handoff verifies original_payload_verified: true and no_action_overlap: true; the latter concerns separation from isolated work, distinct from intended overlap within each contention pair.

Each common measured pair contains one action from each concurrent participant. The following expansion counts describe which recorded action was shorter; they are not independent replications, hypothesis tests or suite scores. Each condition has only two fresh concurrent waves. No pilot timing is pooled into this table.

Pair	Suite	SF	Common measured pairs	Shorter Polars action	Shorter Spark action
A: Spark GPU + Polars CPU 2.0	Baseline	30	12	12	0
A: Spark GPU + Polars CPU 2.0	Baseline	100	12	12	0
A: Spark GPU + Polars CPU 2.0	Complex	30	12	11	1
A: Spark GPU + Polars CPU 2.0	Complex	100	12	10	2
B: Spark CPU + Polars GPU	Baseline	30	12	12	0
B: Spark CPU + Polars GPU	Baseline	100	12	12	0
B: Spark CPU + Polars GPU	Complex	30	8	8	0
B: Spark CPU + Polars GPU	Complex	100	8	8	0

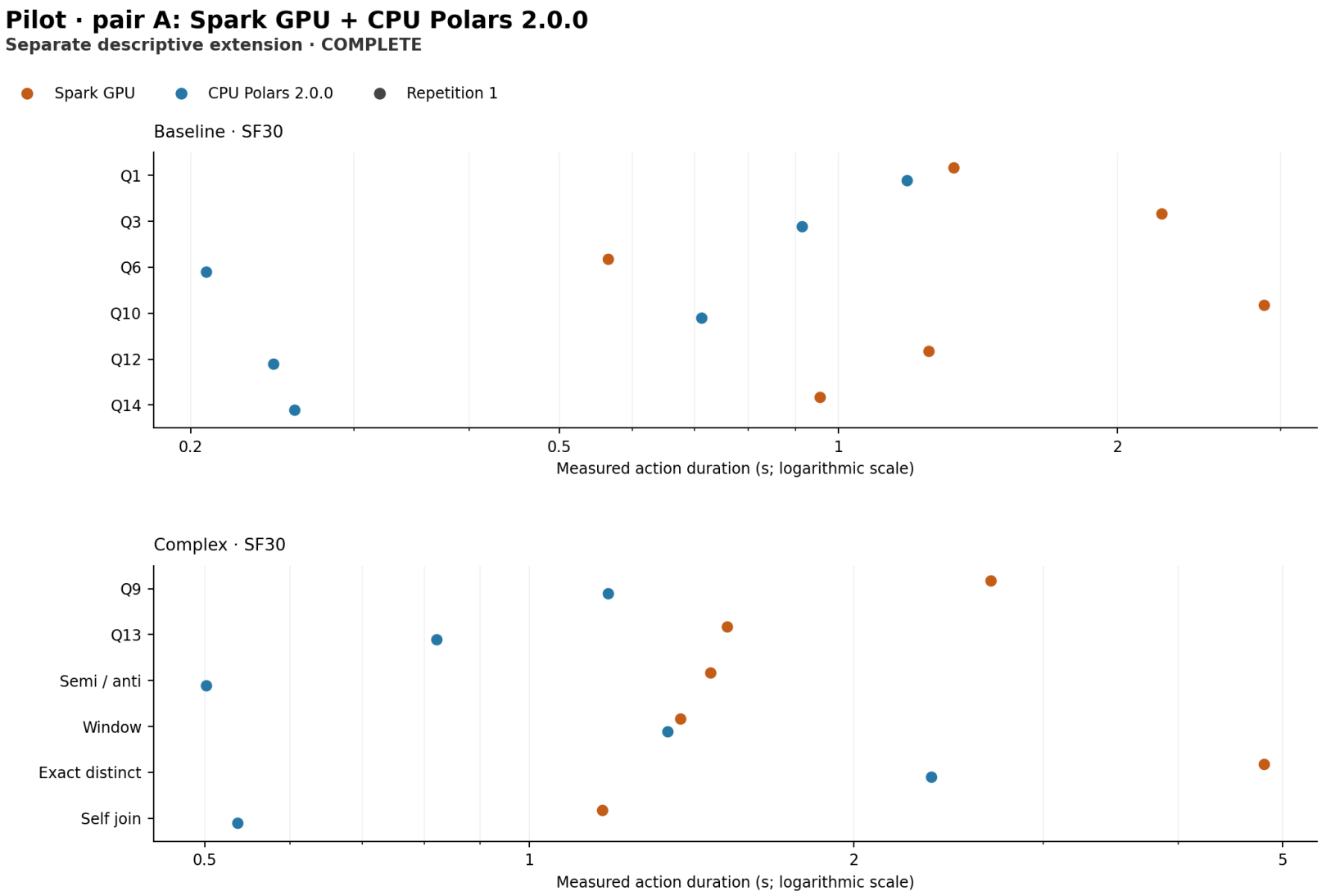
The pair A complex exceptions are the window query: Spark GPU has the shorter action in one SF30 repetition and both SF100 repetitions. For example, its SF100 window actions are 3.176 and 3.243 s, alongside Polars CPU actions of 4.909 and 4.913 s. In the same condition, the Q09 actions are 24.869 and 23.979 s for Spark GPU versus 5.999 and 5.706 s for Polars CPU. The observations therefore vary by query even within one paired condition. In pair B, Polars GPU's shorter recorded actions apply only to supported common expressions; its two complex rejections remain unavailable, with no six-query GPU score. The two pairings have different competitors and cannot rank the GPU engines.

QUERY_ACTIONS_DESCRIPTIVE.csv retains every warmup/measured action, both endpoints and pairing flags; COMMON_QUERY_OVERLAP.csv retains all 110 common measured intersections. Pilot intersections range from 0.208 to 2.361 s with a maximum start skew of 0.647 ms. Expansion intersections range from 0.228 to 8.664 s with a maximum start skew of 2.340 ms. These are observed ranges across different queries, not uncertainty intervals or estimates of synchronization quality beyond these waves.

The six final figures preserve individual actions and actual timelines, separating pilot from expansion. Their manifest SHA256 is 5751b86ee46989fdfc7b0d25b1e2eac3a475a09c872143026cedfbd1e06000bd. Action plots omit unpaired CPU diagnostics; timelines retain them with explicit hatching and use actual UTC intervals and gaps without widening fast actions. No action-time ratio here estimates a contention penalty relative to isolated execution. Such a claim needs a matched isolated analysis with the different replication, preheat, barriers and host history disclosed. Resource counters and sampled memory can include untimed waits and are not action-only costs. No new confidence interval, p value, superiority decision or primary result is inferred from this extension.

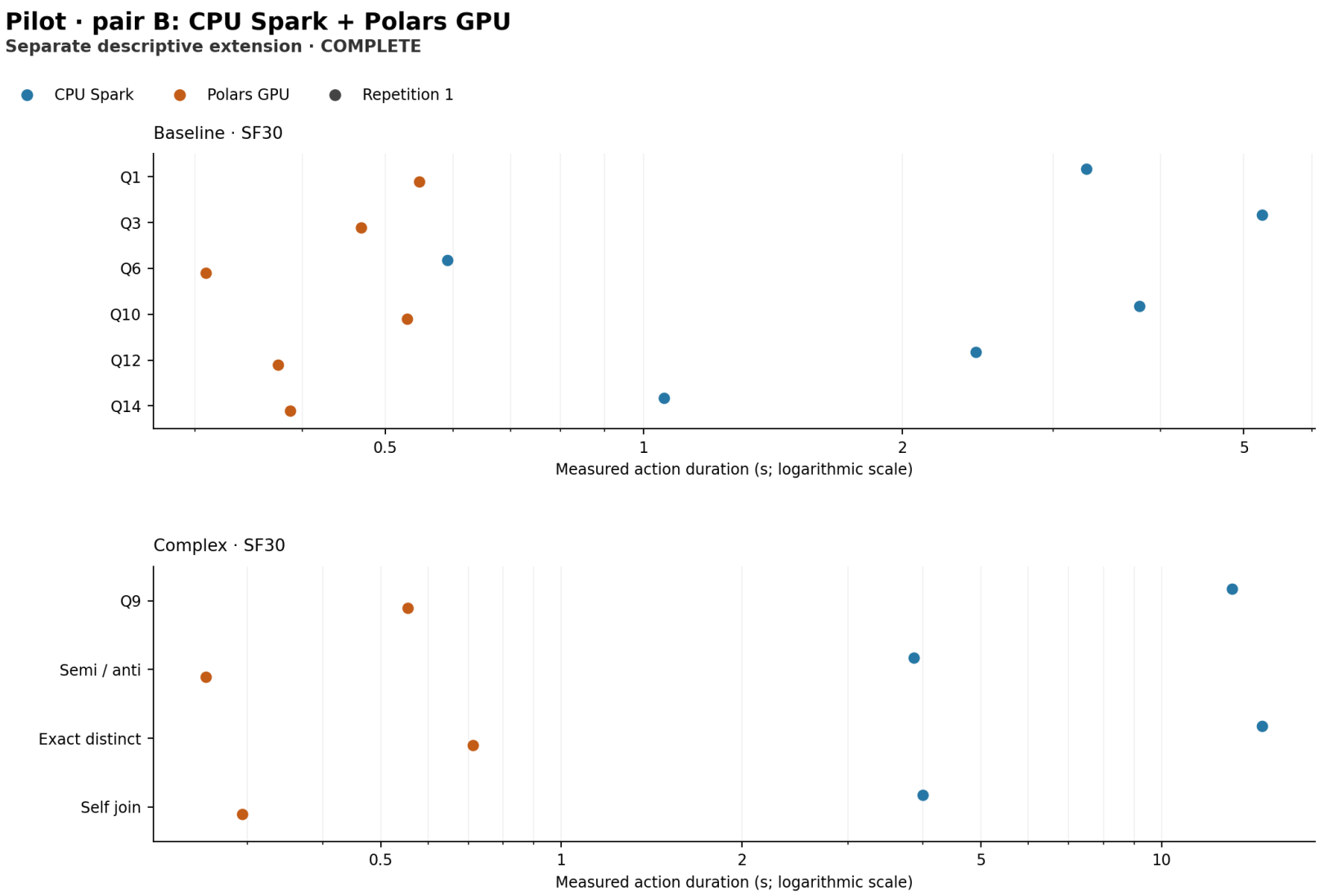
5.8 Separate contention figures

The following exports show the pilot and expansion separately. They retain individual observations, different competitors, unsupported expressions and actual action intervals; they introduce no isolated-versus-contended effect estimate or cross-pair GPU ranking.



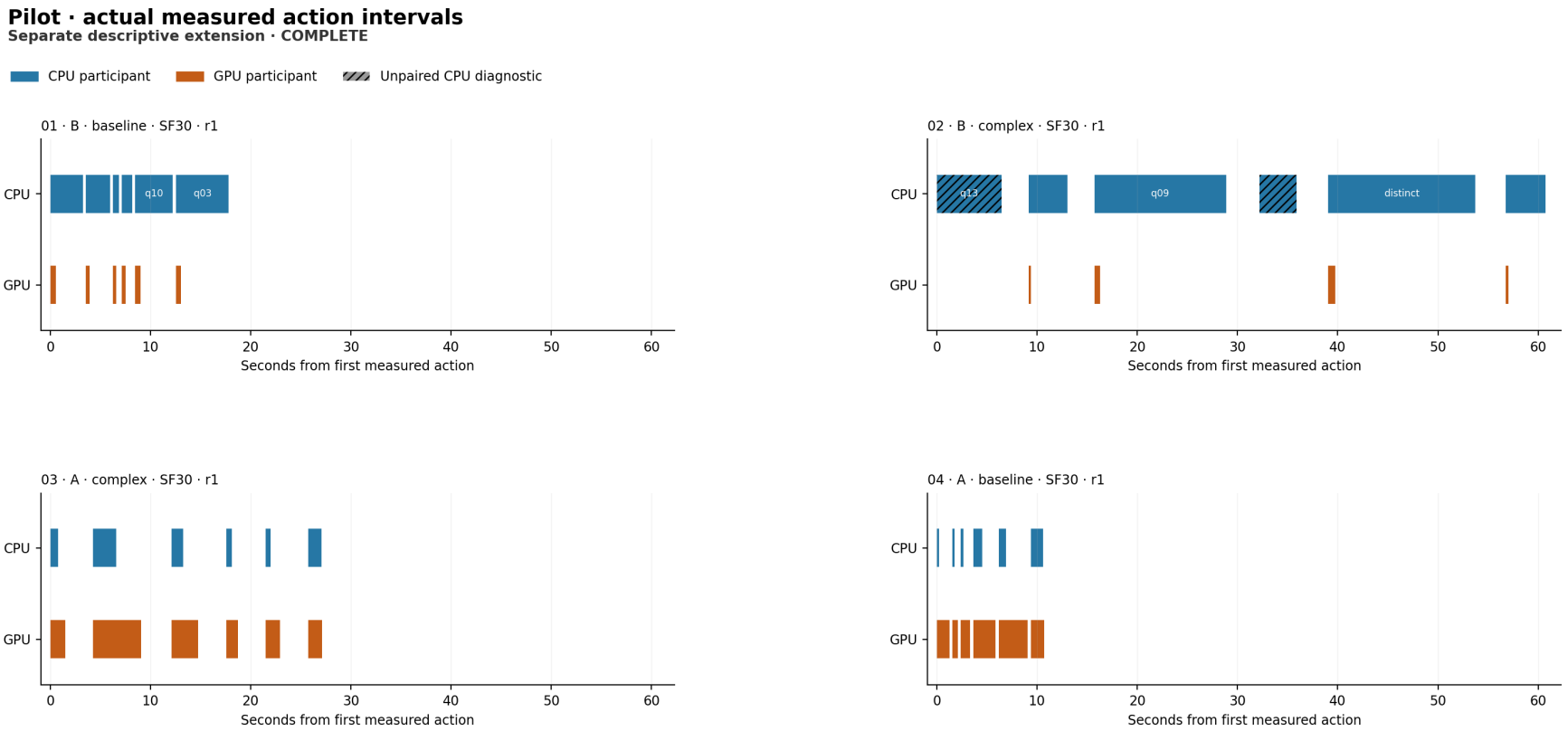
Each point is one measured action from a fresh concurrent wave; no averaging, confidence intervals or tests. Pilot has one repetition at SF30 and is not pooled with expansion. Common queries only. Pair B complex excludes Q13 and window: its CPU-only measurements remain unpaired diagnostics. Pairs have different competitors and do not rank the two GPU engines. Action timers exclude setup and rendezvous waits.

Separate descriptive extension · COMPLETE. Each point is one measured action from a fresh concurrent wave; no averaging, confidence intervals or tests. Pilot has one repetition at SF30 and is not pooled with expansion. Common queries only. Pair B complex excludes Q13 and window: its CPU-only measurements remain unpaired diagnostics. Pairs have different competitors and do not rank the two GPU engines. Action timers exclude setup and rendezvous waits.



Each point is one measured action from a fresh concurrent wave; no averaging, confidence intervals or tests. Pilot has one repetition at SF30 and is not pooled with expansion. Common queries only. Pair B complex excludes Q13 and window: its CPU-only measurements remain unpaired diagnostics. Pairs have different competitors and do not rank the two GPU engines. Action timers exclude setup and rendezvous waits.

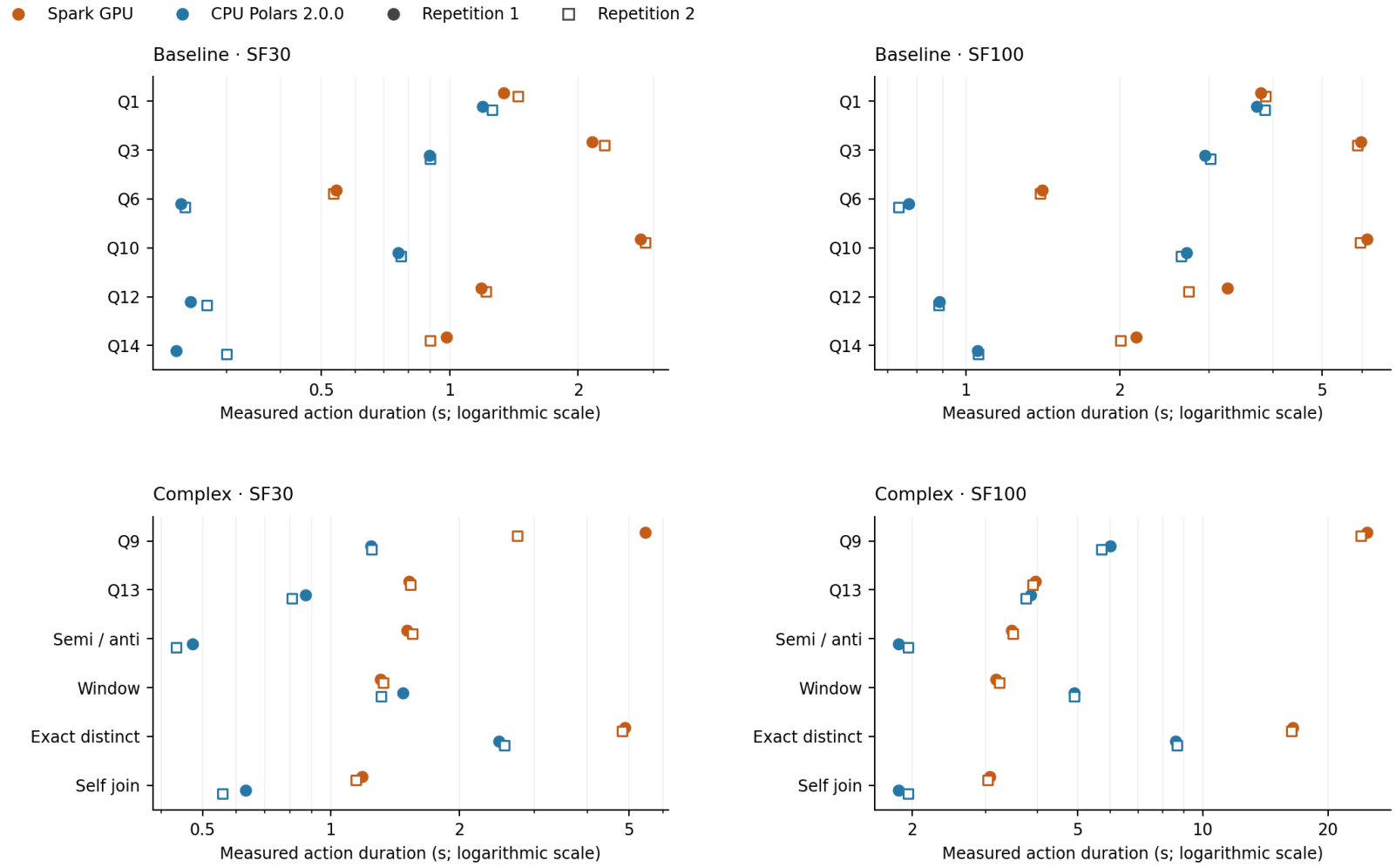
Separate descriptive extension · COMPLETE. Each point is one measured action from a fresh concurrent wave; no averaging, confidence intervals or tests. Pilot has one repetition at SF30 and is not pooled with expansion. Common queries only. Pair B complex excludes Q13 and window: its CPU-only measurements remain unpaired diagnostics. Pairs have different competitors and do not rank the two GPU engines. Action timers exclude setup and rendezvous waits.



Separate descriptive extension · COMPLETE. Every attempted pilot wave appears in planned execution order. Bars retain actual measured UTC start/end intervals and gaps, with a shared linear scale; fast bars are never widened. Zero is each wave’s first measured action, not process startup. CPU/GPU lanes show concurrent participants; warmups are omitted. A: Spark GPU + CPU Polars 2.0.0. B: CPU Spark + Polars GPU. Hatched pair-B Q13/window CPU actions are unpaired diagnostics. Labels appear only where space permits. Both-clock common-query overlap is independently checked in the source audit; the timeline is not a process-lifetime proxy.

Expansion · pair A: Spark GPU + CPU Polars 2.0.0

Separate descriptive extension · COMPLETE

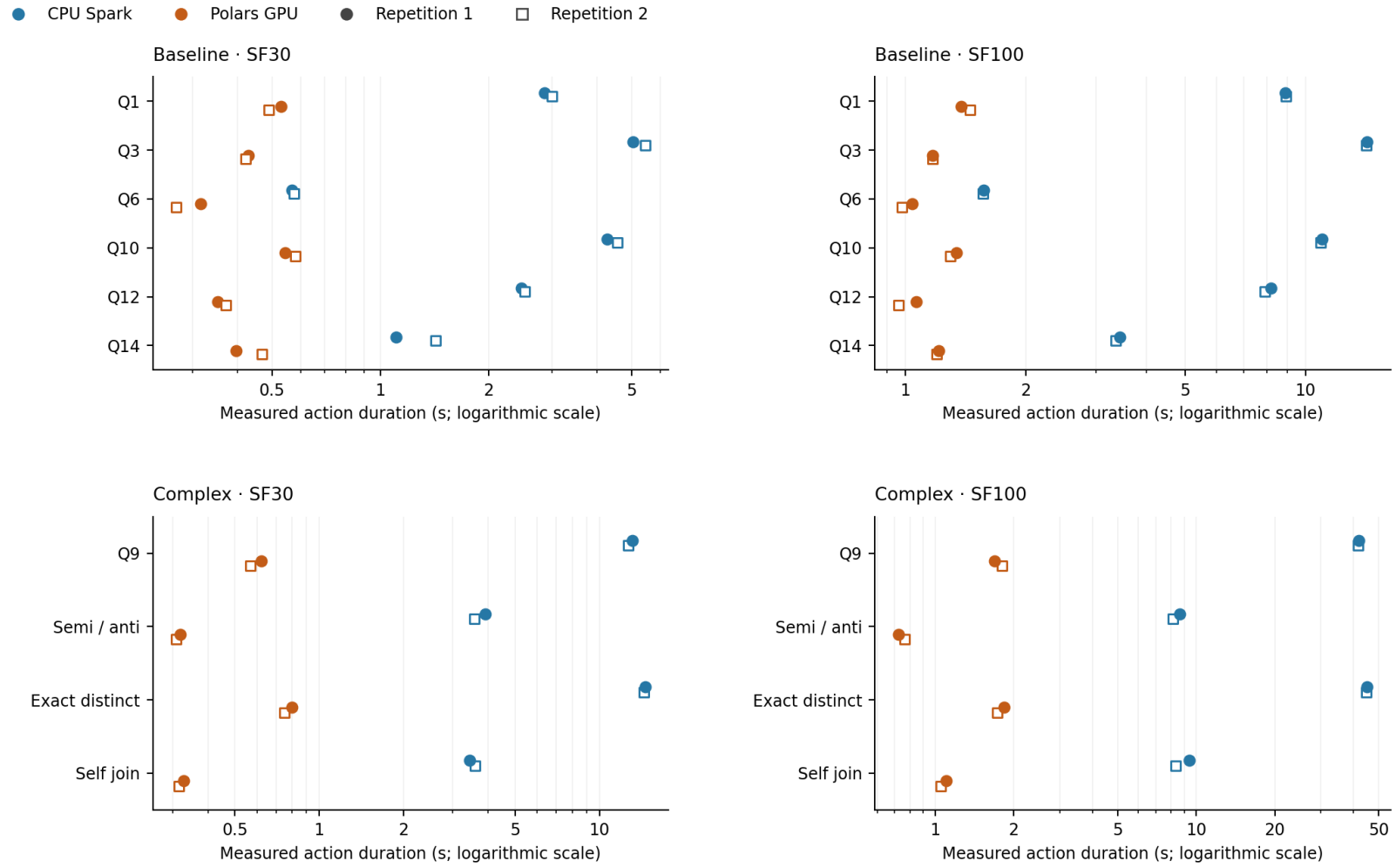


Each point is one measured action from a fresh concurrent wave; no averaging, confidence intervals or tests. Expansion has two repetitions per cell; pilot is shown separately. Common queries only. Pair B complex excludes Q13 and window: its CPU-only measurements remain unpaired diagnostics. Pairs have different competitors and do not rank the two GPU engines. Action timers exclude setup and rendezvous waits.

Separate descriptive extension · COMPLETE. Each point is one measured action from a fresh concurrent wave; no averaging, confidence intervals or tests. Expansion has two repetitions per cell; pilot is shown separately. Common queries only. Pair B complex excludes Q13 and window: its CPU-only measurements remain unpaired diagnostics. Pairs have different competitors and do not rank the two GPU engines. Action timers exclude setup and rendezvous waits.

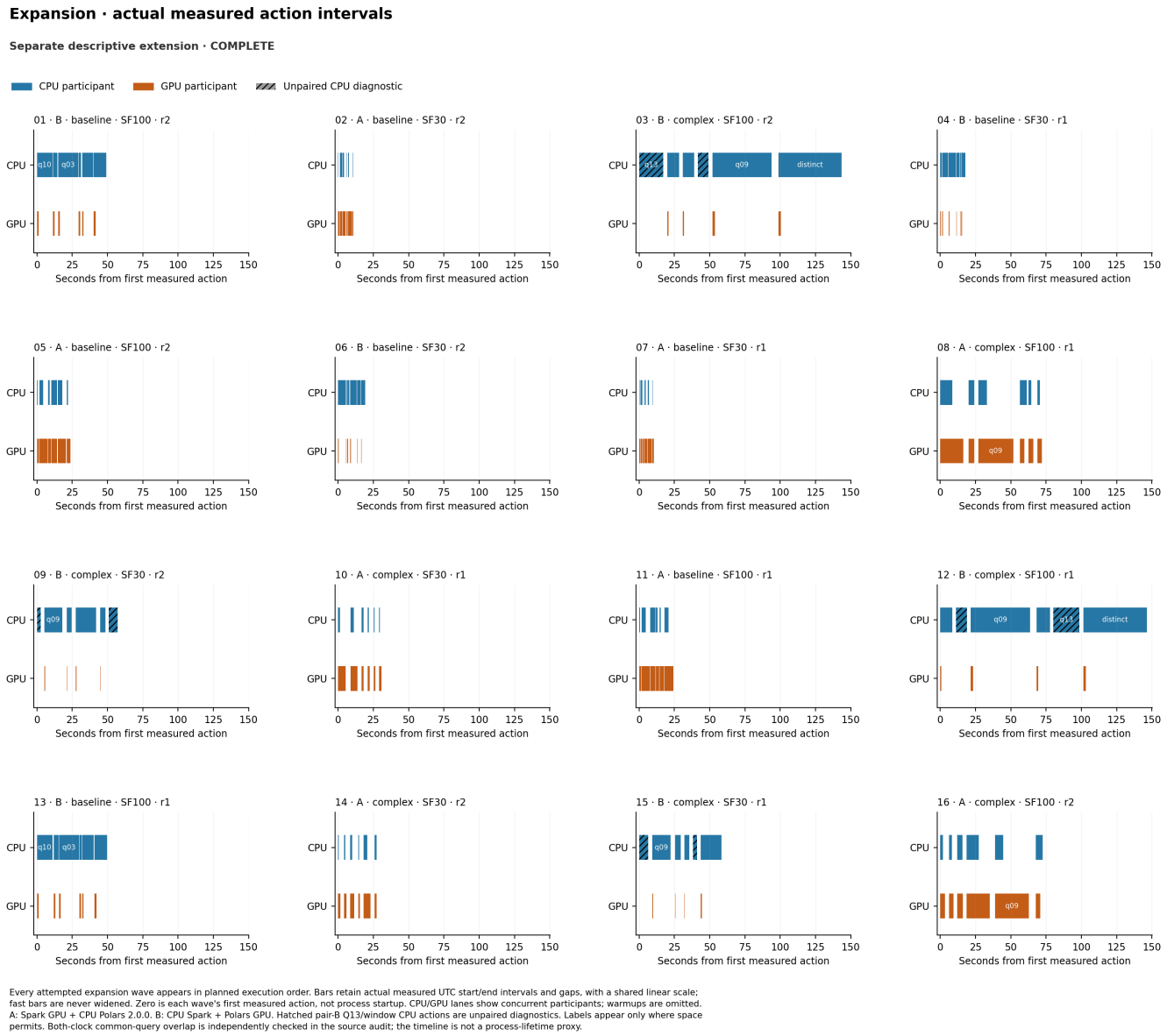
Expansion · pair B: CPU Spark + Polars GPU

Separate descriptive extension · COMPLETE



Each point is one measured action from a fresh concurrent wave; no averaging, confidence intervals or tests. Expansion has two repetitions per cell; pilot is shown separately. Common queries only. Pair B complex excludes Q13 and window: its CPU-only measurements remain unpaired diagnostics. Pairs have different competitors and do not rank the two GPU engines. Action timers exclude setup and rendezvous waits.

Separate descriptive extension · COMPLETE. Each point is one measured action from a fresh concurrent wave; no averaging, confidence intervals or tests. Expansion has two repetitions per cell; pilot is shown separately. Common queries only. Pair B complex excludes Q13 and window: its CPU-only measurements remain unpaired diagnostics. Pairs have different competitors and do not rank the two GPU engines. Action timers exclude setup and rendezvous waits.



Separate descriptive extension · COMPLETE. Every attempted expansion wave appears in planned execution order. Bars retain actual measured UTC start/end intervals and gaps, with a shared linear scale; fast bars are never widened. Zero is each wave's first measured action, not process startup. CPU/GPU lanes show concurrent participants; warmups are omitted. A: Spark GPU + CPU Polars 2.0.0. B: CPU Spark + Polars GPU. Hatched pair-B Q13/window CPU actions are unpaired diagnostics. Labels appear only where space permits. Both-clock common-query overlap is independently checked in the source audit; the timeline is not a process-lifetime proxy.

6. Discussion

The complete-suite and query-level answers differ. Under these fixed settings, GPU Spark reduces the complete-suite score relative to CPU Spark beyond baseline SF1 and throughout the sampled complex workload. It does not reduce the complete-suite score relative to CPU Polars 2.0 at any sampled condition. Nevertheless, its window-query estimates and selected Q1/Q13 estimates are shorter than CPU Polars 2.0. The workload mix therefore matters: a suite-level CPU preference does not imply that every selected operator favors the CPU. These observations identify useful conditions for further profiling without attributing the result to a particular GPU kernel, transfer path or runtime fraction that was not measured.

Strict Polars GPU provides the supported baseline gains from SF10 onward, including a 2.451-fold CPU Polars 2.0/GPU score ratio at SF300. Its unsupported Q13/window expressions limit the complex result more strongly than its fast supported queries can resolve. Users requiring the unchanged full complex workload cannot substitute the four-query GPU subset. GPU Spark supplies complete execution for that workload, while CPU Polars 2.0 has the lower complete-suite score in this experiment.

The two CPU Polars controls answer different configuration questions. CPU 1.44.2 aligns the frontend with the selected GPU integration; CPU 2.0.0 supplies a newer same-Python control. The SF300 baseline ratio is 3.537 against the former and 2.451 against the latter. These are comparisons of recorded software paths and dependencies, not an isolated causal estimate of a frontend-version change. Likewise, Spark and Polars differ in memory policies, execution APIs and dependency inventories. Common hardware does not turn these comparisons into GPU-hardware-only effects or estimates of globally optimal tuning.

Both endpoints retain the same direction for all available complete-suite pairs. This narrows the endpoint concern for these saved output shapes; it does not justify treating action time as application latency. Neither endpoint includes process startup. A sum of query medians is a defined score, not the elapsed time of an application running a complete suite once. The pointwise intervals express ten-block log-ratio uncertainty, while Holm decisions address the one declared family. The small p values do not expand generalization beyond this host, workload and profile.

The cohort history is part of the result. The replacement host changes driver, kernel and physical file generation relative to the lost host; the later recovery reuses replacement inputs while changing the Spark allocation-retry profile after a failure. The completed fresh sample avoids pooling incompatible timings but does not remove configuration-selection history. The passed feasibility diagnostic and successful recovery do not prove a false OOM, a GCLocker root cause, a cure or a causal reliability gain. Allocation-retry and GC time inside measured actions remains part of the normal endpoints.

The separate contention extension describes synchronized pairs requesting the original full CPU budget on one shared host. Its query observations do not rank GPU engines because the competitors differ. Pair B complex also retains two unpaired CPU diagnostics. No matched isolated-versus-contention slowdown analysis is claimed. Verified action separation and scheduler resume do not establish unchanged cache/host conditions; the managed hold and local-power handoff delay remain visible temporal changes in the primary collection.

7. Validity and interpretation limits

One virtual host, possible temporal dependence, VM interference and ten blocks limit generalization and t-model precision. Williams balance does not eliminate all carryover. Fixed settings, unequal memory policies and differing dependency inventories constrain causal software comparisons. Selected queries, Float64 semantics and the preserved window-domain limitation constrain workload and semantic generalization. Unsupported cases have no imputed speed. Mechanistic explanations involving transfers, compute, spilling or bandwidth require separate runtime evidence. The recovery profile was chosen after an observed failure, limiting inference to that post-incident configuration. A fresh complete sample avoids pooling incompatible timings; it does not erase the configuration-selection history or establish a causal mitigation effect. The fixed single-recovery decision and its completed disposition remain part of the interpretation scope. The same-host contention extension is small and synchronized, with only two planned expansion repetitions. Its process-counter and memory observations may span untimed waits; GPU board samples are shared and can miss peaks. The requested interlude changes isolated temporal pacing and may influence later host/cache state. Recorded idle/identity checks and continued preheat policy support interpretation without proving absence of carryover. The separately recorded local low-power event also delayed a between-block off-host receipt while remote block06 completed. Its added temporal spacing may affect later host/cache conditions, without establishing a measured effect or changing any action/worker timing value.

8. Reproducibility and availability

The exact registered recovery campaign, pre-start and execution-profile identities are given in §4.1. The 52-file C84 source remains unchanged. All thirteen original transport snapshots were independently verified and reconciled into a separate harvest with explicit per-file lineage; canonical pre-start bytes and original snapshots remain separate. The reconstruction manifest SHA256 is 6ef171eaaebbda72bfd928b2b12c468a19529e15ec86224d3921c1fcad281b2d. Original JSON bytes retain their recorded paths. Portable replay uses path remapping rather than editing those source artifacts.

The genuine replay completed at 2026-10-28 23:50:00.745311 UTC. Its manifest SHA256 is b59450d0731180b5b9cc642c5174f872ab460ab585445b87230175a07c48655c; the complete audit SHA256 is cb03c639c0b428f50f07a142d1201e8a3c39ad82ea21a2192db95d4ebfc5585d. The final journal SHA256 is 3648cfe093d239af6b6ffd9e14bb6e41926a57f8058604225dc47bf055a27d67. Replay runs the frozen validator, preserves its raw output, records a path-only adapter for Spark plan-source provenance, runs the frozen analyzer, completes the separate configuration/integrity audit and renders the genuine figures. All four subprocesses passed. No GPU, JVM benchmark process or source Parquet read is required for this artifact replay.

The audit binds all 24 idle-boundary profile receipts and 160 effective Spark driver logs to the same sealed profile, Java/runtime hashes and session chronology. It checks all successful outputs, specific unsupported diagnostics, cleanup counts, physical-byte gates and required cache inventories. Its nonsynthetic PASS/no-findings result is an evidence-completeness assessment, not an additional performance hypothesis test.

The claim/evidence matrix and result ledger link exact inputs to all 26 primary rows, forty score/coverage cells, thirty-two secondary pairs and 260 primary block ratios. Machine-readable artifacts preserve full precision. Appendix B and C retain secondary endpoints and raw primary block ratios; the four primary figures are copied from the bound replay exports without altering estimates. Separate contention raw archives, terminal audit, full handoff and six final figures retain their own identities and do not satisfy any isolated session requirement.

Data and code availability: the frozen source, complete raw evidence, transport snapshots, reconstruction lineage and derived replay artifacts are retained in a private research archive. Raw archive access is controlled by the author; this paper does not imply a public raw-data or source-code release. The paper provides complete summary tables and figures, with exact artifact hashes for verification. Workload provenance and software references are identified below; these selected research workloads are not certified TPC-H results. Daniel Ejiofor is the author; no institutional affiliation is asserted.

9. Conclusion

A complete ten-block, 400-session comparison on the recorded H100 host supports GPU gains for specific configuration pairs and scales. GPU Spark beats CPU Spark from baseline SF10 onward and at all three complex scales, but CPU Polars 2.0 has the lower complete-suite score at every sampled condition. GPU Spark still has shorter descriptive estimates for the window query and selected Q1/Q13 conditions. Strict Polars GPU beats both CPU Polars versions on the baseline suite from SF10 onward, while its two unsupported complex expressions prevent any complete complex-suite score.

All 26 prespecified tests reject after Holm adjustment, with fifteen GPU-favoring and eleven reference-favoring directions. Available common-host- result comparisons preserve direction. These findings apply to the fixed post-incident JVM profile, tested input domain and recorded host/pacing history; they establish no general GPU-hardware effect, exact unmeasured crossover, causal mitigation or contention slowdown. The separately completed contention extension remains descriptive and excluded from primary inference.

Appendix A. Historical evidence excluded from recovery inference

A.1 Earlier pre-start correctness evidence

The lost October 26 campaign’s archived readiness passes with 116 successful comparisons across ten SF1 smoke sessions. All configurations pass core inner/outer/semi-anti cases. CPU configurations and GPU Spark pass all five diagnostics; Polars GPU passes three and rejects rank/window and DOTALL-regex cases. The tiny Spark GPU final plan contains LeftAnti. This establishes small-input semantics/selection, not anti-join performance or a nonempty anti branch in the original correlated workload. These facts are historical; replacement/recovery readiness was independently repeated and archived for its own cohort, as reported in §4.1.

Eight complete independent oracles are archived for the earlier measured conditions. Its separate post-hoc pilot supplement compares all 936 saved successful outputs in 40 historical sessions to those SQL references, with zero mismatches under the stated tolerance rule. This strengthens the pilot’s correctness record; it does not retroactively preregister its timings or add independent timing repetitions to the replacement.

A.2 Exploratory pilot performance context

At SF300, the pilot’s baseline scores were 12.774 s for Polars GPU, 31.540 s for its historical Polars 2.0/Python 3.10 CPU control, and 45.528 s for GPU Spark. Its complex scores were 70.389 s for that CPU control and 143.039 s for GPU Spark; Polars GPU had no complete complex score. These are sums of six per-query medians from one session per configuration/condition. They are descriptive historical observations without process-level confidence intervals. Historical Python 3.10 timings do not populate the replacement same-Python CPU control or its prospective results.

Primary literature and official implementation references are collected in the bibliography below.

Appendix B. All secondary endpoint pairs

These thirty-two rows retain all four configuration pairs at every measured condition. Each available row has ten paired blocks; the six Polars GPU complex rows have zero complete blocks and N/A estimates/intervals. B denotes baseline, C complex. Prefixes and contrast IDs use the same rule as Table 2. Values are reference/GPU ratios with pointwise 95% intervals. Every available direction- change flag is false; unavailable comparisons have no such flag. No secondary hypothesis test is performed.

B. CPU Polars 2.0 / GPU Spark

Prefix polars_cpu_200_over_spark_gpu.

Condition	Action ratio [95% CI]	Host-result ratio [95% CI]	Direction change
B/1	0.055945 [0.054808, 0.057105]	0.056103 [0.054965, 0.057265]	No
B/10	0.212549 [0.208690, 0.216479]	0.212647 [0.208785, 0.216581]	No
B/30	0.388511 [0.382310, 0.394813]	0.388567 [0.382366, 0.394869]	No
B/100	0.591820 [0.580946, 0.602897]	0.591842 [0.580966, 0.602921]	No
B/300	0.698245 [0.686991, 0.709683]	0.698254 [0.687000, 0.709692]	No
C/30	0.467006 [0.431938, 0.504921]	0.467039 [0.431972, 0.504954]	No
C/100	0.523867 [0.515150, 0.532732]	0.523873 [0.515156, 0.532738]	No
C/300	0.492635 [0.478992, 0.506668]	0.492637 [0.478994, 0.506669]	No

B. CPU Spark / GPU Spark

Prefix spark_cpu_over_spark_gpu.

Condition	Action ratio [95% CI]	Host-result ratio [95% CI]	Direction change
B/1	0.736771 [0.685003, 0.792450]	0.736787 [0.685025, 0.792460]	No
B/10	1.528723 [1.451437, 1.610125]	1.528691 [1.451411, 1.610086]	No
B/30	2.396972 [2.300420, 2.497576]	2.396925 [2.300381, 2.497521]	No
B/100	3.118779 [3.055933, 3.182918]	3.118748 [3.055905, 3.182884]	No
B/300	3.296495 [3.239099, 3.354908]	3.296483 [3.239086, 3.354896]	No
C/30	3.975933 [3.678199, 4.297766]	3.975507 [3.677842, 4.297263]	No
C/100	3.521337 [3.472958, 3.570391]	3.521231 [3.472850, 3.570287]	No
C/300	2.605614 [2.556563, 2.655606]	2.605594 [2.556545, 2.655584]	No

B. CPU Polars 1.44.2 / Polars GPU

Prefix polars_cpu_144_over_polars_gpu.

Condition	Action ratio [95% CI]	Host-result ratio [95% CI]	Direction change
B/1	0.401941 [0.390084, 0.414158]	0.402791 [0.390931, 0.415011]	No
B/10	1.491644 [1.474820, 1.508659]	1.491864 [1.475029, 1.508891]	No
B/30	2.554085 [2.513373, 2.595456]	2.554047 [2.513355, 2.595398]	No
B/100	3.187462 [3.125726, 3.250418]	3.187412 [3.125680, 3.250363]	No
B/300	3.536788 [3.473146, 3.601596]	3.536763 [3.473122, 3.601571]	No
C/30	N/A	N/A	N/A
C/100	N/A	N/A	N/A
C/300	N/A	N/A	N/A

B. CPU Polars 2.0 / Polars GPU

Prefix polars_cpu_200_over_polars_gpu.

Condition	Action ratio [95% CI]	Host-result ratio [95% CI]	Direction change
B/1	0.372908 [0.365387, 0.380583]	0.373832 [0.366307, 0.381512]	No
B/10	1.280664 [1.265771, 1.295732]	1.280971 [1.266093, 1.296023]	No
B/30	1.954230 [1.918831, 1.990282]	1.954278 [1.918890, 1.990318]	No
B/100	2.372428 [2.320547, 2.425469]	2.372422 [2.320539, 2.425466]	No
B/300	2.451064 [2.405104, 2.497903]	2.451062 [2.405102, 2.497900]	No
C/30	N/A	N/A	N/A
C/100	N/A	N/A	N/A
C/300	N/A	N/A	N/A

Appendix C. All 260 primary block ratios

Each row contains the ten original paired score ratios in chronological block order, split into blocks 01-05 and 06-10 for readability. Ratios above one favor the denominator GPU. B denotes baseline, C complex. No observation is trimmed or promoted from inner actions. The final column is the analyzer’s sample standard deviation of block log ratios. Display rounding does not alter the full-precision raw CSV, estimates or tests.

C. CPU Polars 2.0 / GPU Spark

Prefix polars_cpu_200_over_spark_gpu.

Condition	Blocks 01-05	Blocks 06-10	SD(log ratio)
B/1	0.054063, 0.053971, 0.055719, 0.054146, 0.058632	0.057650, 0.055877, 0.055740, 0.057407, 0.056452	0.028692
B/10	0.215024, 0.202382, 0.207919, 0.217156, 0.213920	0.206890, 0.214846, 0.219111, 0.217686, 0.211178	0.025611
B/30	0.395199, 0.401695, 0.386869, 0.384770, 0.402371	0.378507, 0.385338, 0.379086, 0.391375, 0.380791	0.022492
B/100	0.618452, 0.617574, 0.575760, 0.593368, 0.598619	0.583303, 0.586346, 0.589722, 0.575713, 0.581139	0.025924
B/300	0.714452, 0.709582, 0.694008, 0.659544, 0.709835	0.703856, 0.700667, 0.705643, 0.694495, 0.691968	0.022714
C/30	0.537869, 0.425208, 0.410391, 0.508820, 0.413406	0.504791, 0.491621, 0.497294, 0.502238, 0.403222	0.109119
C/100	0.543660, 0.545121, 0.513816, 0.507641, 0.514115	0.528068, 0.517334, 0.520681, 0.526083, 0.523459	0.023457
C/300	0.509055, 0.543185, 0.486766, 0.471604, 0.491981	0.485654, 0.485750, 0.482175, 0.489604, 0.484071	0.039262

C. CPU Spark / GPU Spark

Prefix spark_cpu_over_spark_gpu.

Condition	Blocks 01-05	Blocks 06-10	SD(log ratio)
B/1	0.741002, 0.828107, 0.640948, 0.832348, 0.783368	0.750979, 0.645271, 0.696457, 0.666928, 0.816548	0.101842
B/10	1.385768, 1.528677, 1.571238, 1.334650, 1.692699	1.505768, 1.621462, 1.501497, 1.626154, 1.555042	0.072522
B/30	2.520604, 2.619520, 2.307546, 2.352377, 2.626103	2.305448, 2.385720, 2.278130, 2.233508, 2.376803	0.057474
B/100	3.110696, 3.184578, 3.058913, 3.095247, 3.339951	3.022086, 3.089980, 3.130072, 3.041960, 3.125821	0.028457
B/300	3.393600, 3.292255, 3.253117, 3.116730, 3.394407	3.283167, 3.314862, 3.308136, 3.350535, 3.267018	0.024553
C/30	4.470605, 3.663226, 3.439022, 4.292144, 3.482143	4.206460, 4.274886, 4.403158, 4.260007, 3.476870	0.108807
C/100	3.517463, 3.594915, 3.545761, 3.476678, 3.539327	3.650396, 3.397398, 3.500960, 3.496736, 3.499669	0.019339

Condition	Blocks 01-05	Blocks 06-10	SD(log ratio)
C/300	2.645298, 2.676885, 2.595402, 2.484677, 2.735584	2.597053, 2.602963, 2.552902, 2.559945, 2.613717	0.026567

C. CPU Polars 1.44.2 / Polars GPU

Prefix polars_cpu_144_over_polars_gpu.

Condition	Blocks 01-05	Blocks 06-10	SD(log ratio)
B/1	0.400257, 0.395735, 0.407849, 0.395027, 0.411838	0.444925, 0.399975, 0.391239, 0.380285, 0.395506	0.041857
B/10	1.538215, 1.448493, 1.486887, 1.489769, 1.501365	1.480161, 1.508776, 1.485460, 1.504017, 1.474984	0.015856
B/30	2.619186, 2.621674, 2.515631, 2.553846, 2.644609	2.521852, 2.538134, 2.554014, 2.461788, 2.515920	0.022462
B/100	3.336093, 3.337681, 3.156181, 3.150904, 3.127925	3.109042, 3.159279, 3.233560, 3.188369, 3.086386	0.027341
B/300	3.603691, 3.527948, 3.422343, 3.553108, 3.437838	3.526619, 3.637225, 3.512336, 3.703602, 3.453455	0.025383

C. CPU Polars 2.0 / Polars GPU

Prefix polars_cpu_200_over_polars_gpu.

Condition	Blocks 01-05	Blocks 06-10	SD(log ratio)
B/1	0.362049, 0.366898, 0.372836, 0.356355, 0.390534	0.385444, 0.368173, 0.369242, 0.380264, 0.378645	0.028481
B/10	1.293475, 1.266002, 1.265692, 1.255747, 1.315376	1.260364, 1.304095, 1.301997, 1.273458, 1.271980	0.016351
B/30	1.960146, 2.007904, 1.946052, 1.948911, 2.056975	1.879625, 1.966359, 1.923969, 1.903699, 1.954431	0.025554
B/100	2.483902, 2.505675, 2.365012, 2.330760, 2.350356	2.323908, 2.356211, 2.369079, 2.393728, 2.255895	0.030909
B/300	2.572266, 2.503877, 2.447094, 2.392706, 2.411307	2.543350, 2.425765, 2.385693, 2.434968, 2.401390	0.026461

References

These references support methodological or implementation context. Experimental facts and numerical findings require the separately archived study evidence ledger. Documentation was consulted on 2026-10-26; the related-work synthesis rechecked the primary Kent repository, IBM publication record, NIST handbook and Hesterberg preprint abstract on 2026-10-27. Installed pinned source and recorded effective settings govern runtime claims where rolling documentation differs.

1. **Kalibera, T., and Jones, R. E. (2013).** Rigorous Benchmarking in Reasonable Time. *Proceedings of ISMM*, 63-74. DOI; author manuscript, University of Kent. Supports separating repetition levels, reporting uncertainty, and allocating replication according to variation and experiment cost. It does not establish a universal repetition count or prove that this campaign's ten blocks have adequate power.

2. **Mytkowicz, T., Diwan, A., Hauswirth, M., and Sweeney, P. F. (2009).** Producing Wrong Data Without Doing Anything Obviously Wrong! *Proceedings of ASPLOS*. Primary publication record, IBM Research. Supports caution about setup bias and experimental allocation. It does not demonstrate the absence of bias on this virtual host.

3. **NIST/SEMATECH.** e-Handbook of Statistical Methods, section 5.3.3.2, Randomized Block Designs. Official handbook. Supplies general blocking/randomization principles. The exact Williams sequences and balance checks are part of this study's frozen protocol and executable schedule tests.

4. **Holm, S. (1979).** A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2), 65-70. Publisher archive record. Source of the familywise multiple-testing procedure applied to the single prespecified 26-test family. This does not make the reported pointwise confidence intervals simultaneous or remove the assumptions behind input p values.

5. **Hesterberg, T. C. (2015).** What Teachers Should Know About the Bootstrap: Resampling in the Undergraduate Statistics Curriculum. *The American Statistician*, 69(4), 371-386. DOI; author preprint. Motivates caution about small-sample bootstrap coverage and the distinction between original observations and computational resamples. The manuscript's log-ratio t model remains an assumption-dependent prespecified choice.

6. **Transaction Processing Performance Council.** TPC Benchmark H Standard Specification, revision 3.0.1. Official specification. Defines the complete query/refresh workload, standard timing and metrics. This selected-query Float64 study does not claim those metrics, compliance or certification. Notice and provenance records are retained separately in the private research archive.

7. **DuckDB.** Version 1.5 documentation, `tpch` extension. Official documentation; tagged 1.5.6 SQL query source. Documents generator use, deviations from official generated data and fixed substitution parameters. Generator/version provenance does not establish benchmark compliance.

8. **Polars.** GPU Support, user guide. Official documentation. Describes GPU collection and `raise_on_fail=True`. This study additionally saves effective configuration and positive/unsupported session canaries; documentation alone does not prove execution mode.

9. **Apache Spark.** Spark 3.5.9 SQL Performance Tuning. Official versioned documentation. Describes AQE, caching, partition and join configuration, supporting retention of actual settings and final plans. It is not a prescription that the chosen fixed settings are optimal for every engine.

10. **NVIDIA.** cuDF plugin for Apache Spark, version 26.08.1 compatibility. Pinned official source. Documents ordering and floating-point differences relevant to comparison semantics. Exact result checks and selected operators still require saved study artifacts.

11. **NVIDIA.** cuDF-Polars profiling and tracing. Official versioned documentation. Distinguishes runtime profiling from plan interpretation. The current manuscript makes no kernel-time, transfer-time, or GPU-runtime-fraction claim. APIs must be checked against installed 26.10.2 before any additional diagnostic run.

12. **OpenJDK.** JDK-8192647, GCLocker allocation issue record. Primary issue tracker. The prospective recovery protocol cites this record for allocation-starvation behavior and discussion of retry count as a possible workaround with longer-stall risk. It supplies a plausible mechanism, not a diagnosis of this study’s Java heap-space error, proof of a false OOM or a causal mitigation result.

13. **OpenJDK.** JEP 423. Official JEP. Addresses G1 JNI critical-region handling in JDK 22. The recovery retains Java 17.0.20.1; citing this JEP does not imply that its implementation is present in the study runtime or that the study upgrades Java.

14. **Apache Spark.** Pull request 51920, closed unmerged. Primary proposal and disposition. The recovery protocol identifies this proposal’s retry-count-100 option as context for the separately gated fixed-profile diagnostic. A closed unmerged PR is not official Spark 3.5.9 tuning guidance, a validated cure or evidence that the profile improves performance or reliability. The fixed recovery configuration and retained actual driver flags determine the study’s scope.